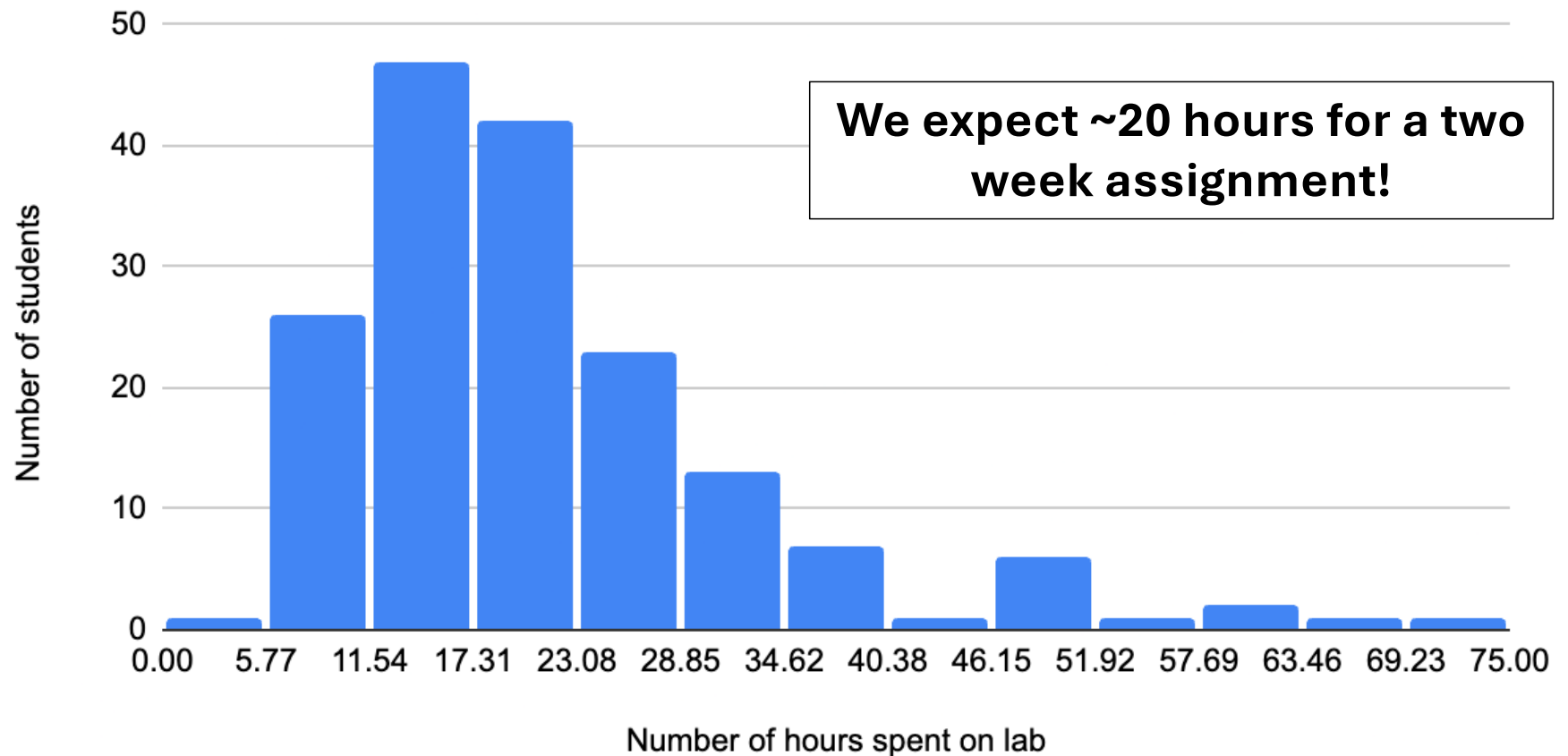




# Announcements

- shell lab due apr 8
- malloc lab (final) reflection due april 1  
<https://forms.gle/xWqhYRC6jTGtBkxw6>
- SCS building facilities says HVAC issue should be fixed now

On average students spent 21 hours on Malloc Lab  
(Checkpoint)



# Clarifications from last lecture...

# System level: below standard level

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("output.txt", "w");
    if (!fp) {
        perror("output.txt");
        return 1;
    }
    fputs("baby shark (do doo dooo)\n", fp);
    if (fclose(fp)) {
        perror("output.txt");
        return 1;
    }
    return 0;
}
```

```
.globl close
close:
    mov $3, %eax
    syscall
    cmp $-4096, %rax
    jae __syscall_error
    ret
```

```
FILE *fopen(const char *fname,
             const char *mode) {
    int fd = open(fname,
                  __mode2flags(mode),
                  DEFFILEPERMS);

    if (fd == -1) {
        return NULL;
    }
    return fdopen(fd, mode);
}
```

```
int fputs(const char *s, FILE *fp) {
    size_t n = strlen(s);
    while (n > 0) {
        ssize_t written =
            write(fp->fd, s, n);
        if (written < 0) return EOF;
        n -= written;
        s += written;
    }
    return 0;
}
```

```
int fclose(FILE *fp) {
    int rv = close(fp->fd);
    __ffree(fp);
    return rv;
}
```

# Writing Files

**Writing a file copies bytes from memory to the current file position, and then updates current file position**

```
1 char buf[512];
2 int fd;      /* file descriptor */
3 int nbytes;  /* number of bytes read */
4
5
6 /* Open the file fd ... */
7 /* Then write up to 512 bytes from buf to file fd */
8 if ((nbytes = write(fd, buf, sizeof(buf)) < 0) {
9     perror("write");
10    exit(1);
11 }
```

**Returns number of bytes written from `buf` to file `fd`**

`nbytes < 0` indicates that an error occurred

As with reads, short counts are possible and are not errors!

# Today: How do computer networks work?

- **Network types and structures**
- Locating a host
- Setting up a connection
- HTTP Example

**We have a lot to cover today!**

# Many things we won't cover

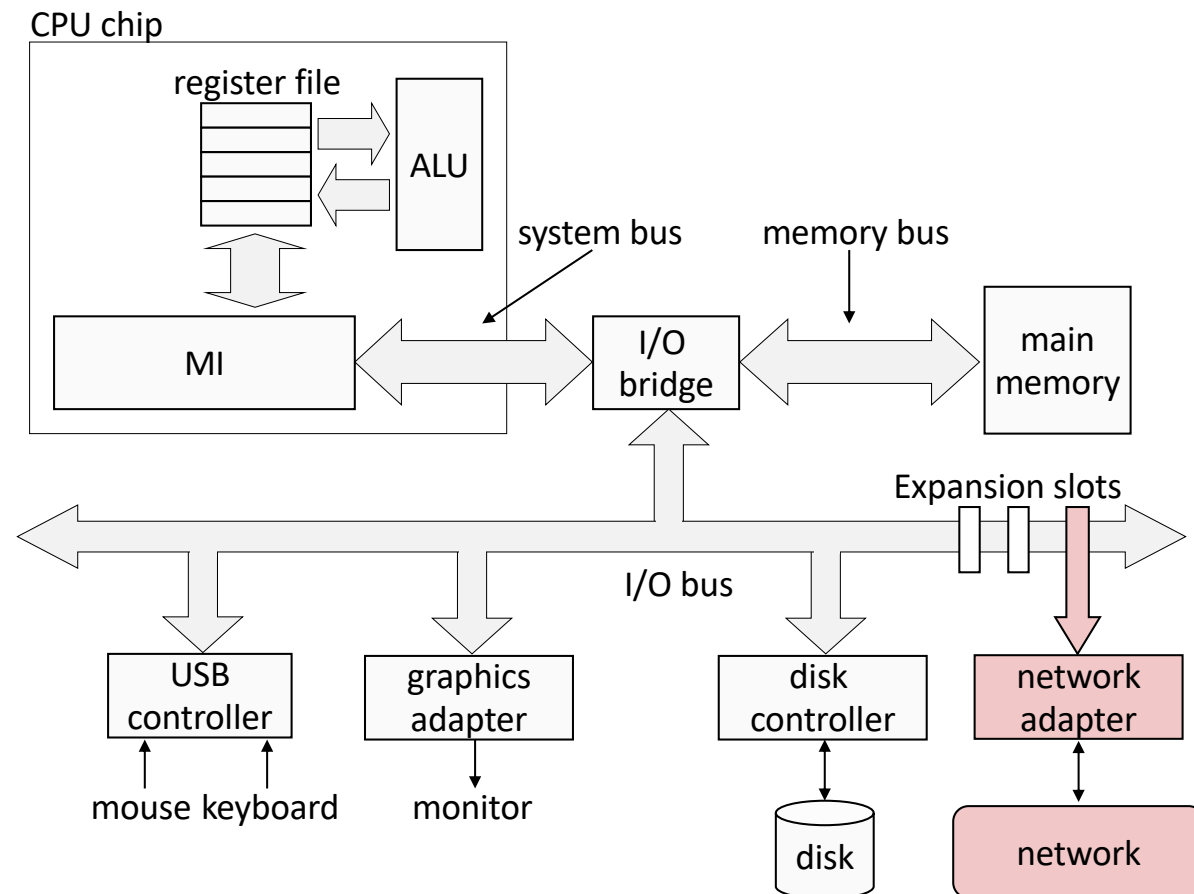
- How does Wi-Fi work?
- What is a CDN?
- What is congestion control?
- What is a middlebox?
- How does Google search work?
- How does a VPN work?
- Etc...

What happens when you got to a website?

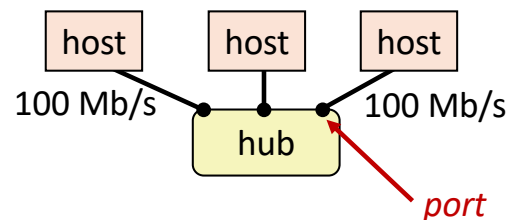
# What is a computer network?

- A *network* is a hierarchical system of boxes and wires organized by geographical proximity
  - LAN (Local Area Network) spans a building or campus
    - Ethernet is most prominent example
  - WAN (Wide Area Network) spans country or world
    - Typically high-speed point-to-point (mostly optical) links
  - Also: SAN (Storage area network), MAN (Metropolitan), etc., etc.
- An *internetwork* (*internet*) is an interconnected set of networks
  - The Global IP Internet (uppercase “I”) is the most famous example of an internet (lowercase “i”)
- Let’s see how an internet is built from the ground up

# Hardware Organization of a Network Host

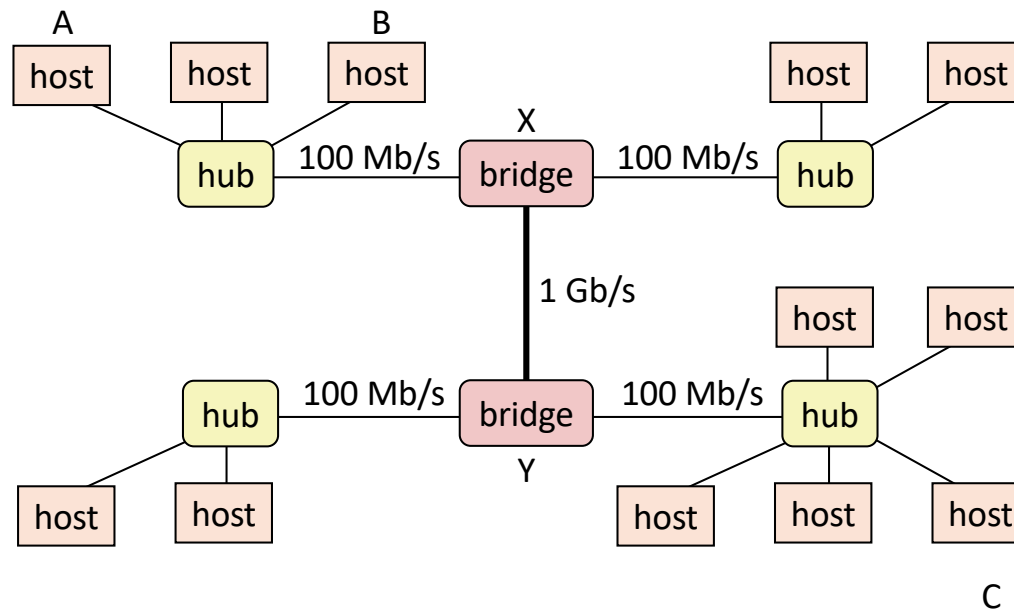


# How to connect computers in the same room or building? Ethernet!



- Ethernet segment consists of a collection of *hosts* connected by wires (twisted pairs) to a *hub*
- Spans room or floor in a building
- Operation
  - Each Ethernet adapter has a unique 48-bit address (MAC address)
    - E.g., 00:16:ea:e3:54:e6

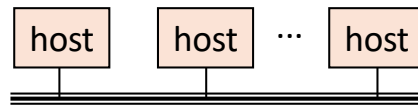
## Next Level: Bridged Ethernet Segment



- Spans building or campus
- Bridges cleverly learn which hosts are reachable from which ports and then selectively copy frames from port to port

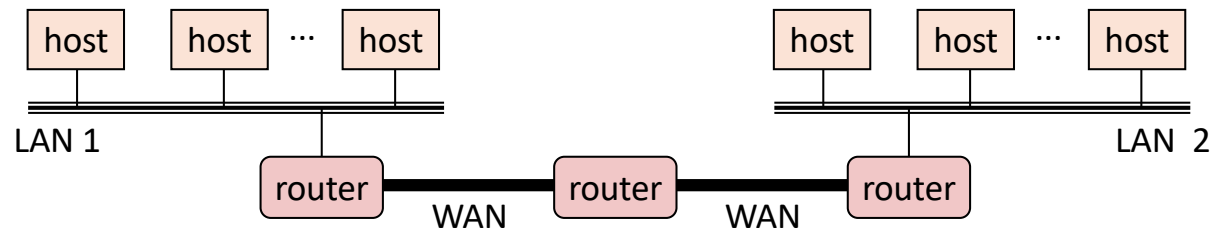
# Conceptual View of LANs

- For simplicity, hubs, bridges, and wires are often shown as a collection of hosts attached to a single wire:



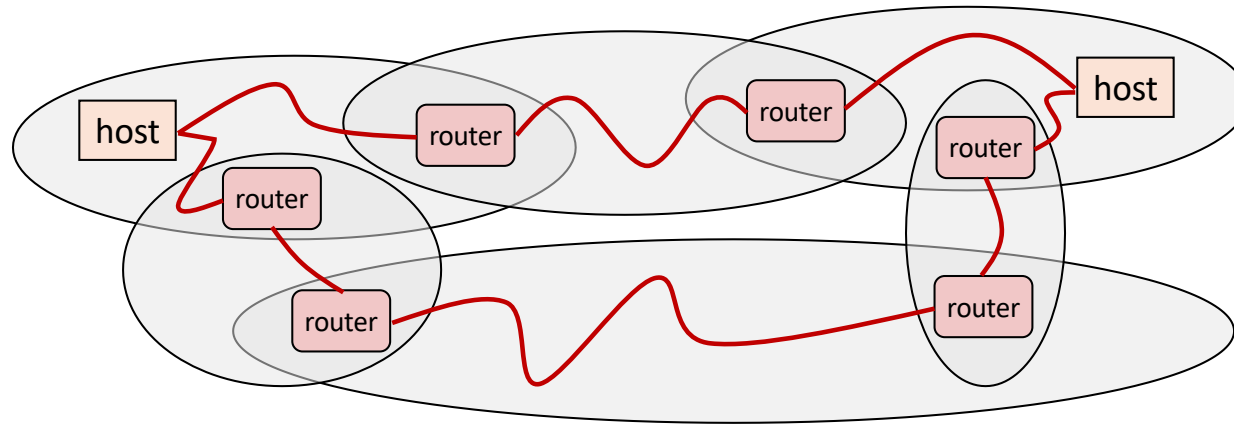
## Next Level: internets

- Multiple incompatible LANs can be physically connected by specialized computers called *routers*
- The connected networks are called an *internet* (lower case)



*LAN 1 and LAN 2 might be completely different, totally incompatible  
(e.g., Ethernet, Fibre Channel, 802.11\*, T1-links, DSL, ...)*

# Logical Structure of an internet



- Ad hoc interconnection of networks
  - No particular topology
  - Vastly different router & link capacities
- Send packets from source to destination by hopping through networks
  - Router forms bridge from one network to another
  - Different packets may take different routes

# The Notion of an internet Protocol

- How is it possible to send bits across incompatible LANs and WANs?
- Solution: *protocol* software running on each host and router
  - Protocol is a set of rules that governs how hosts and routers should cooperate when they transfer data from network to network.
  - Smooths out the differences between the different networks

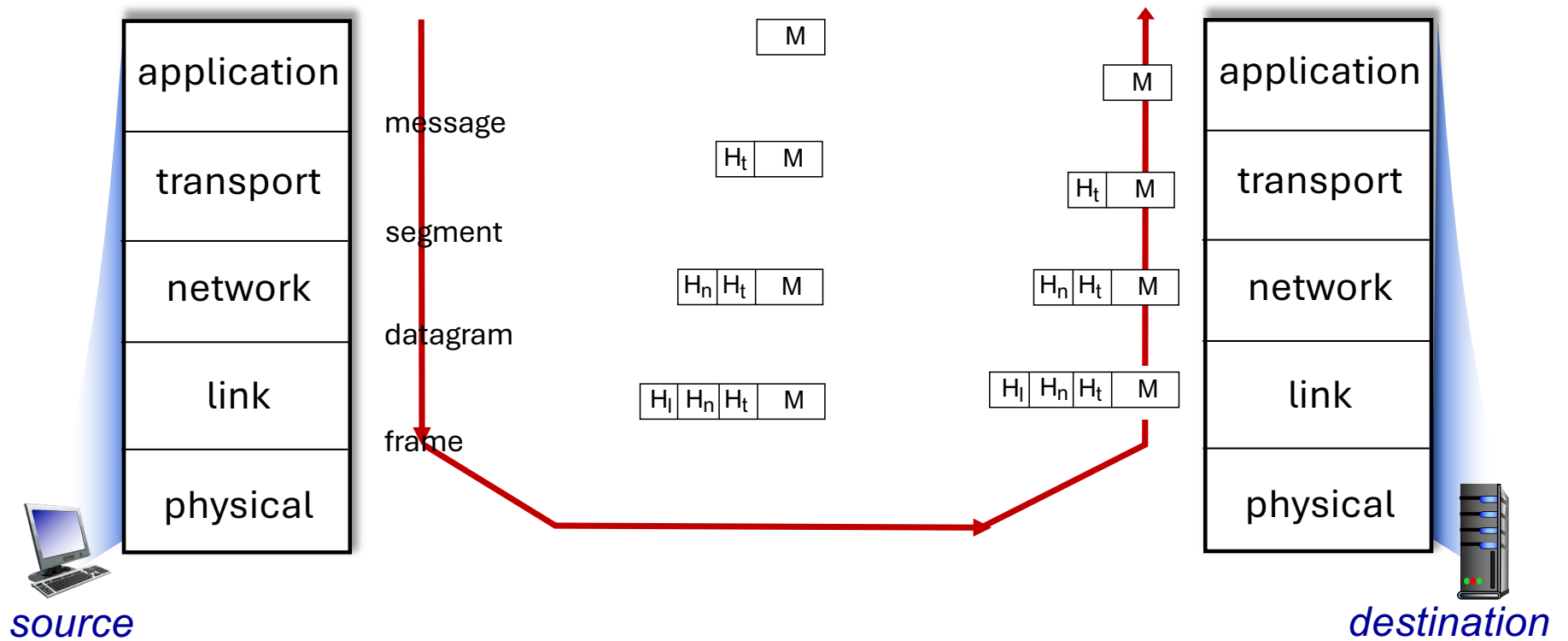
# What Does an internet Protocol Do?

- Provides a *naming scheme*
  - An internet protocol defines a uniform format for **host addresses**
  - Each host (and router) is assigned at least one of these internet addresses that uniquely identifies it
- Provides a *delivery mechanism*
  - An internet protocol defines a standard transfer unit (**packet**)
  - Packet consists of **header** and **payload**
    - Header: contains info such as packet size, source and destination addresses
    - Payload: contains data bits sent from source host

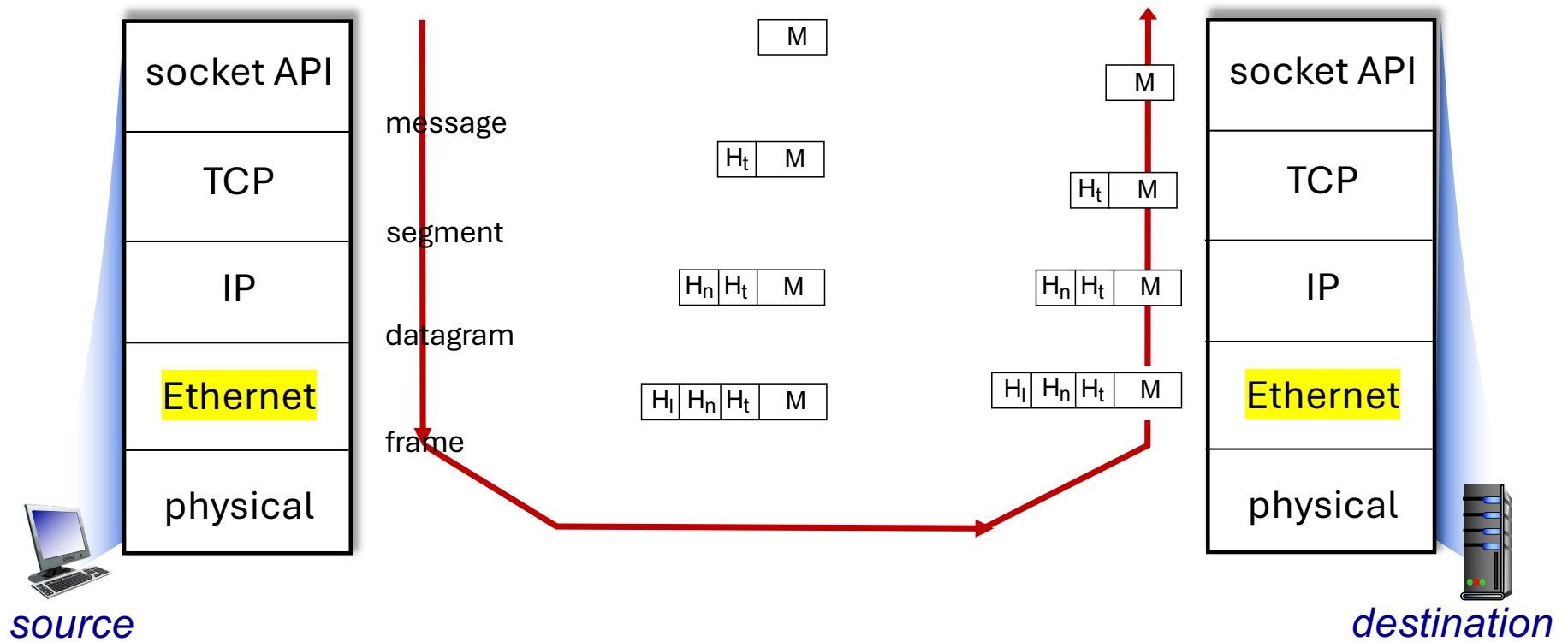
# Why packets?

- Packet switching: transferring data from source to destination by breaking it up into smaller pieces called packets
- Pros:
  - If packet not received then a client can ask a server to resend it
  - Easier to handle corrupted and out-of-order data
  - Different packets can take different routes for adapting to changing network conditions
- Cons:
  - Increases total number of bytes that must be transferred
  - Splitting data into packets may not be suitable for something that requires low response time like video calling

# Services, Layering and Encapsulation



# Services, Layering and Encapsulation



# Other Issues

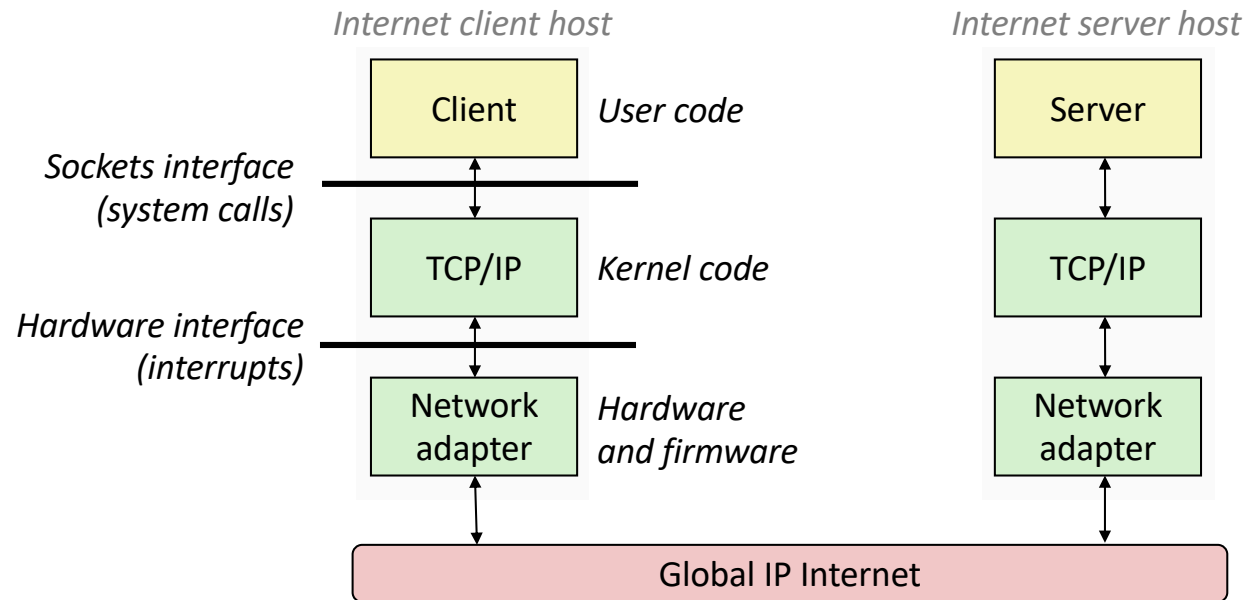
- We are glossing over a number of important questions:
  - What if different networks have different maximum frame sizes? (segmentation)
  - How do routers know where to forward frames?
  - How are routers informed when the network topology changes?
  - What if packets get lost?
- These (and other) questions are addressed by the area of systems known as *computer networking*



# Global IP Internet (upper case)

- Most famous example of an internet
- Based on the TCP/IP protocol family
  - IP (Internet Protocol)
    - Provides *basic naming scheme* and unreliable *delivery capability* of packets (datagrams) from *host-to-host*
  - UDP (User Datagram Protocol)
    - Uses IP to provide *unreliable* datagram delivery from *process-to-process*
  - TCP (Transmission Control Protocol)
    - Uses IP to provide *reliable* byte streams from *process-to-process* over *connections*
- Accessed via a mix of Unix file I/O and functions from the *sockets interface*

# Hardware and Software Organization of an Internet Application



# Today: How do computer networks work?

- Network types and structures
- **Locating a host**
- Setting up a connection
- HTTP Example

# A Programmer's View of the Internet

1. Hosts are mapped to a set of 32-bit *IP addresses*

- 128.2.203.179
- 127.0.0.1 (always *localhost*)

2. As a convenience for humans, the Domain Name System maps a set of identifiers called Internet *domain names* to IP addresses:

- [www.cs.cmu.edu](http://www.cs.cmu.edu) “resolves to” 128.2.217.3

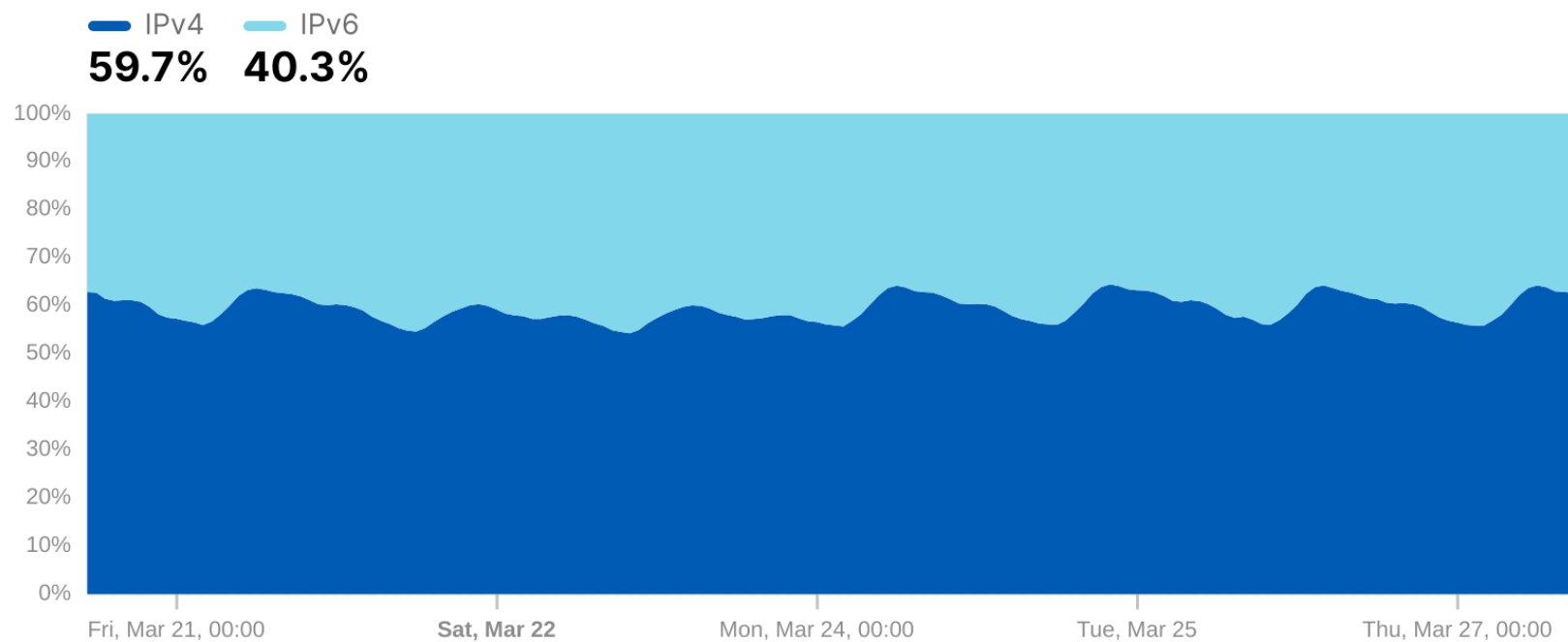
3. A process on one Internet host can communicate with a process on another Internet host over a *connection*

## Aside: IPv4 and IPv6

- *IPv4* (Internet Protocol version 4) specified 1981
  - 32-bit host addresses (192.0.2.43)
  - Known to not be enough for everyone since ~1990
- IPv6 (Internet Protocol version 6) specified 1996
  - 128-bit addresses (2001:0db8:0:0:0:0:cafe:la7e)
  - Intended to replace IPv4
  - Very slow adoption due to need to replace routers (CMU's network doesn't support IPv6 at all!)
- Application programmers mostly don't have to care
  - Sockets API makes it easy to write code that seamlessly uses either, as necessary

## IPv4 vs. IPv6 Worldwide

Distribution of human traffic by IP version



Log into milkshark machine and run:

```
traceroute sandshark.ics.cs.cmu.edu
```

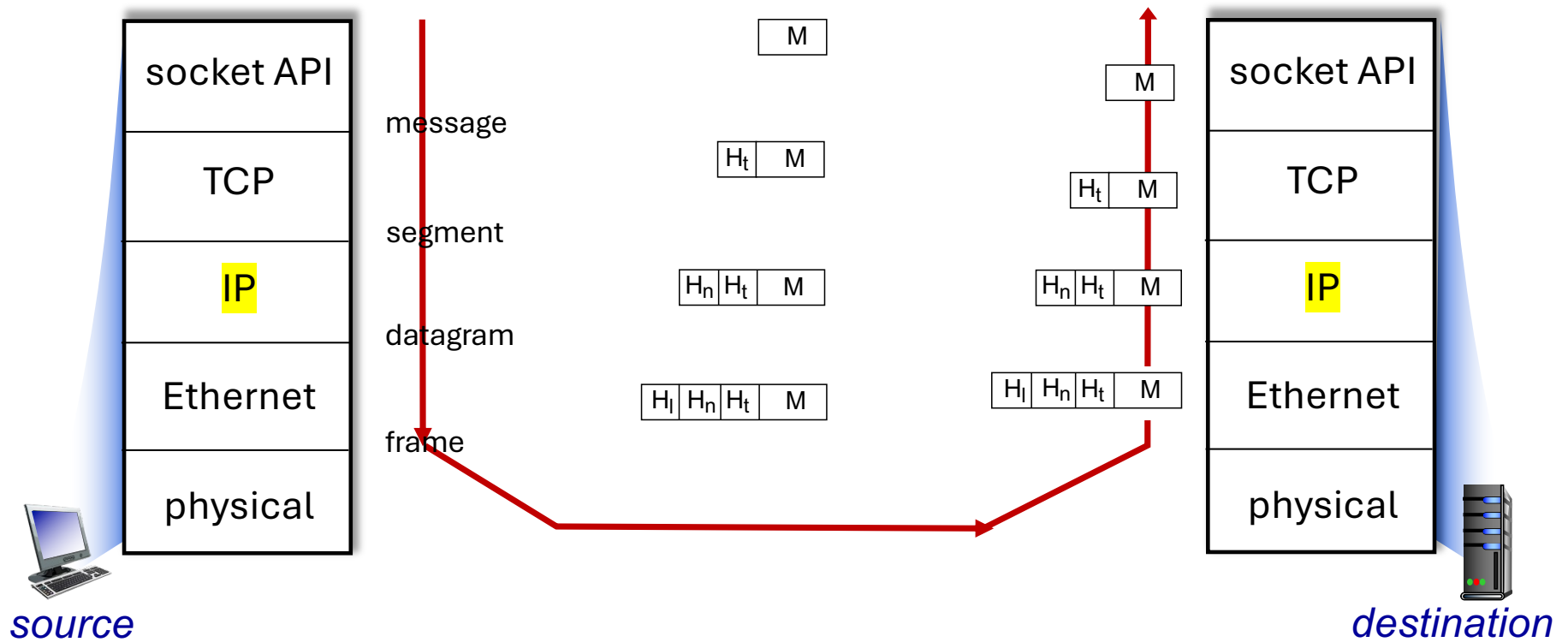
How many routers?

Now try running...

```
traceroute cs.cmu.edu
```

How many routers? What do you notice about the names of the routers?

# Services, Layering and Encapsulation



# (1) IP Addresses

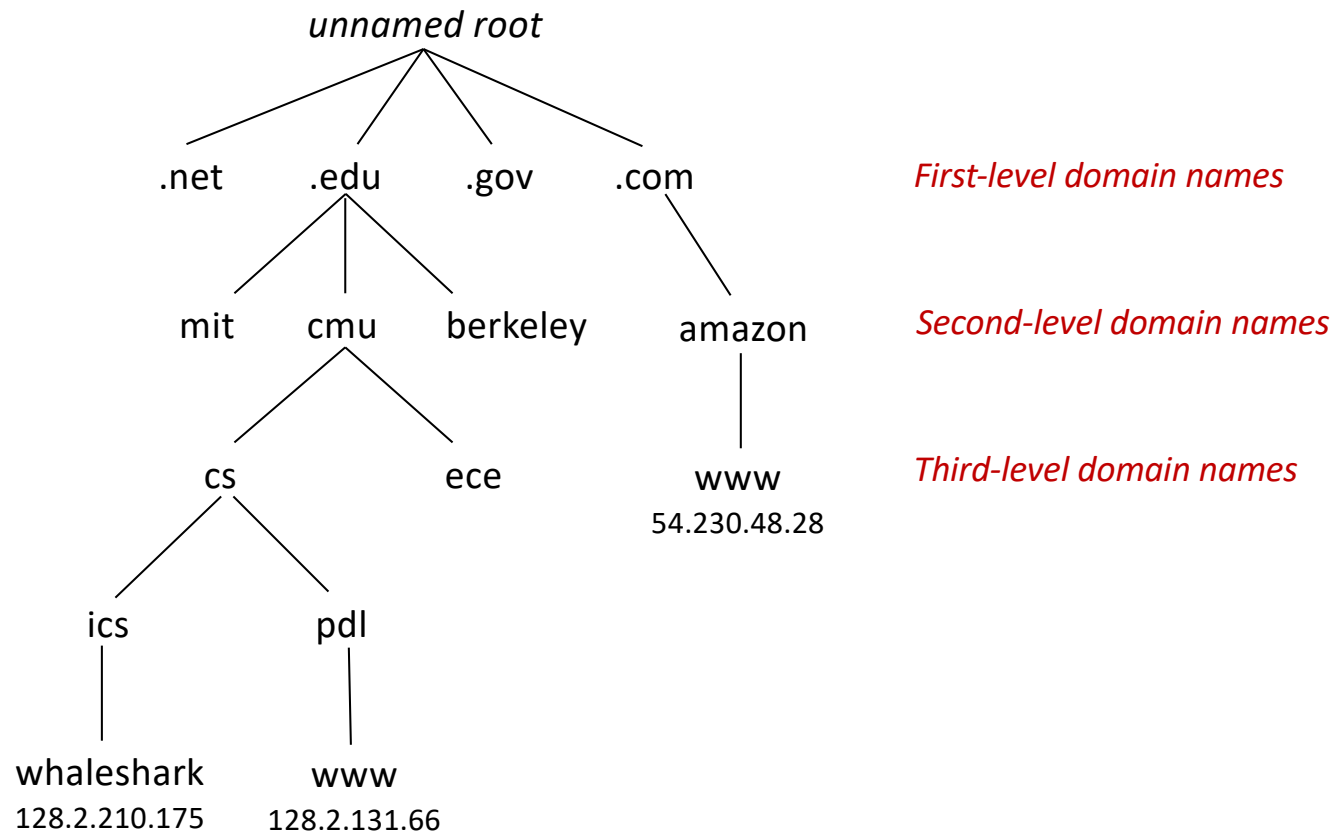
- 32-bit IP addresses are stored in an *IP address struct*
  - IP addresses are always stored in memory in *network byte order* (big-endian byte order)
  - True in general for any integer transferred in a packet header from one machine to another.
    - E.g., the port number used to identify an Internet connection.

```
1  /* Internet address structure */
2  struct in_addr {
3      uint32_t  s_addr; /* network byte order (big-endian) */
4  };
```

# Dotted Decimal Notation

- By convention, each byte in a 32-bit IP address is represented by its decimal value and separated by a period
  - IP address: `0x8002C2F2` = `128.2.194.242`
- Use `getaddrinfo` and `getnameinfo` functions (described next lecture) to convert between IP addresses and dotted decimal format.

## (2) Internet Domain Names



```
nslookup cs.cmu.edu
```

```
nslookup andrew.cmu.edu
```

What do the output addresses have in common?

# Domain Naming System (DNS)

- The Internet maintains a mapping between IP addresses and domain names in a worldwide distributed database called *DNS*
- Conceptually, programmers can view the DNS database as a collection of millions of *host entries*.
  - Each host entry defines the mapping between a set of domain names and IP addresses.

# Properties of DNS Mappings

- Can explore properties of DNS mappings using `nslookup`

- (Output edited for brevity)

```
linux> nslookup localhost
Address: 127.0.0.1
```

- Each host has a locally defined domain name `localhost` which always maps to the *loopback address* `127.0.0.1`

```
linux> hostname
whaleshark.ics.cs.cmu.edu
```

- Use `hostname` to determine real domain name of local host:

# Properties of DNS Mappings (cont)

- Simple case: one-to-one mapping between domain name and IP address:

```
linux> nslookup whaleshark.ics.cs.cmu.edu
Address: 128.2.210.175
```

- Multiple domain names mapped to the same IP address:

```
linux> nslookup cs.mit.edu
Address: 18.25.0.23
linux> nslookup eecs.mit.edu
Address: 18.25.0.23
```

- And backwards: 

```
linux> nslookup 18.25.0.23
23.0.25.18.in-addr.arpa  name = eecs.mit.edu.
```

# Properties of DNS Mappings (cont)

- Multiple domain names mapped to multiple IP addresses:

```
linux> nslookup www.twitter.com
Address: 104.244.42.65
Address: 104.244.42.129
Address: 104.244.42.193
Address: 104.244.42.1
```

```
linux> nslookup www.twitter.com
Address: 104.244.42.129
Address: 104.244.42.65
Address: 104.244.42.193
Address: 104.244.42.1
```

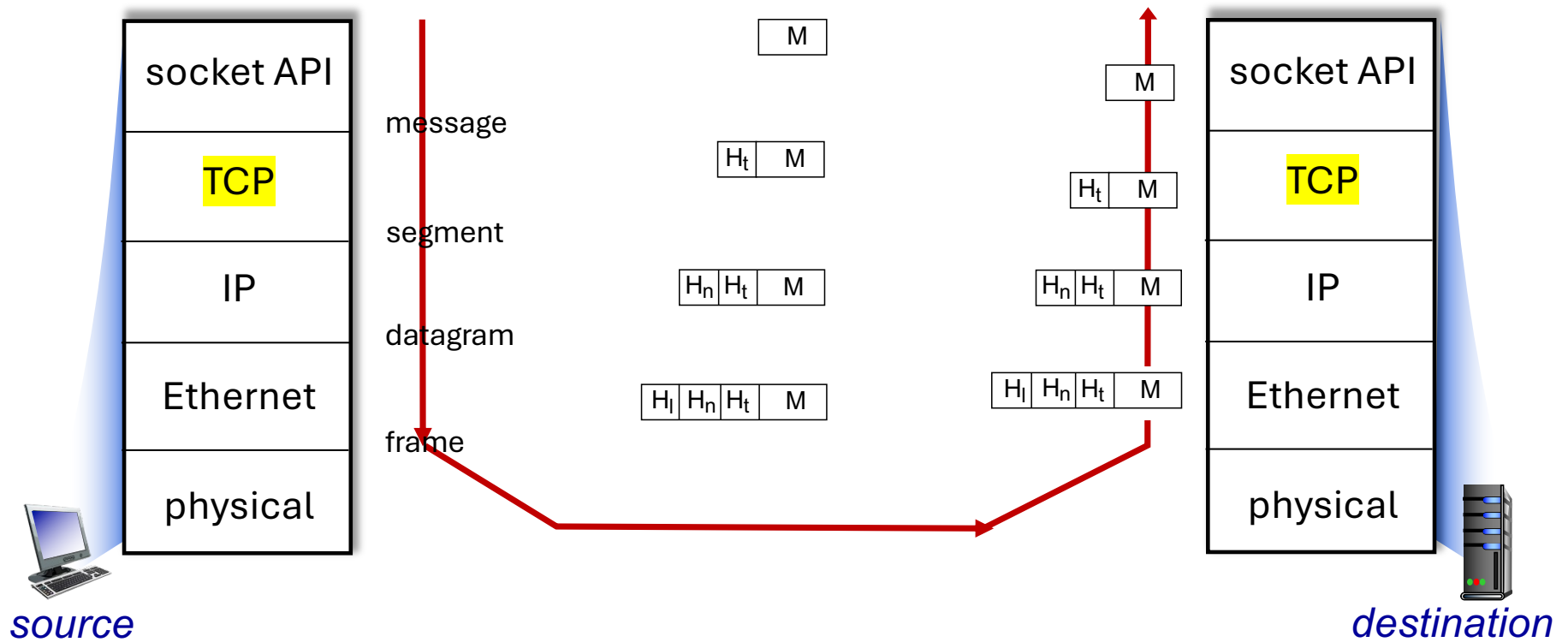
- Some valid domain names don't map to any IP address:

```
linux> nslookup ics.cs.cmu.edu
(No Address given)
```

# Today: How do computer networks work?

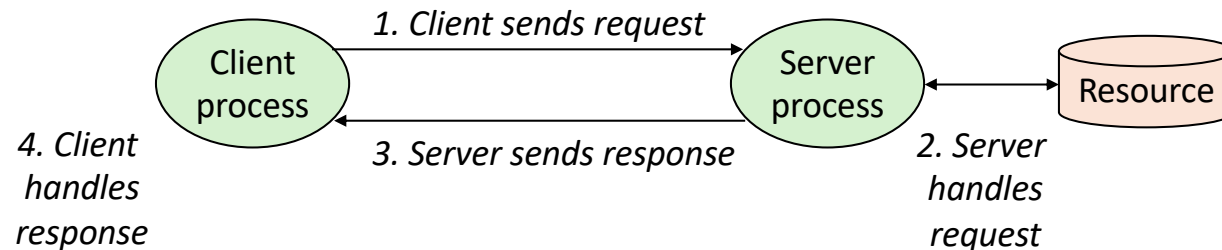
- Network types and structures
- Locating a host
- **Setting up a connection**
- HTTP Example

# Services, Layering and Encapsulation



# A Client-Server Transaction

- Most network applications are based on the client-server model:
  - A **server** process and one or more **client** processes
  - Server manages some **resource**
  - Server provides **service** by manipulating resource for clients
  - Server activated by request from client (vending machine analogy)



*Note: clients and servers are processes running on hosts  
(can be the same or different hosts)*

## (3) Internet Connections

- Clients and servers most often communicate by sending streams of bytes over TCP *connections*. Each connection is:
  - *Point-to-point*: connects a pair of processes.
  - *Full-duplex*: data can flow in both directions at the same time,
  - *Reliable*: stream of bytes sent by the source is eventually received by the destination in the same order it was sent.
- A *socket* is an endpoint of a connection
  - *Socket address* is an **IPaddress:port** pair
- A *port* is a 16-bit integer that identifies a process:
  - **Ephemeral port**: Assigned automatically by client kernel when client makes a connection request.
  - **Well-known port**: Associated with some *service* provided by a server (e.g., port 80 is associated with Web servers)

# Well-known Service Names and Ports

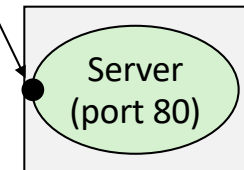
- Popular services have permanently assigned *well-known ports* and corresponding *well-known service names*:
  - echo servers: echo 7
  - ftp servers: ftp 21
  - ssh servers: ssh 22
  - email servers: smtp 25
  - Unencrypted Web servers: http 80
  - SSL/TLS encrypted Web: https 443
- Mappings between well-known ports and service names is contained in the file `/etc/services` on each Linux machine.

# Anatomy of a Connection

- A connection is uniquely identified by the socket addresses of its endpoints (*socket pair*)
  - `(cliaddr:cliport, servaddr:servport)`

Server socket address

208.216.181.15:80



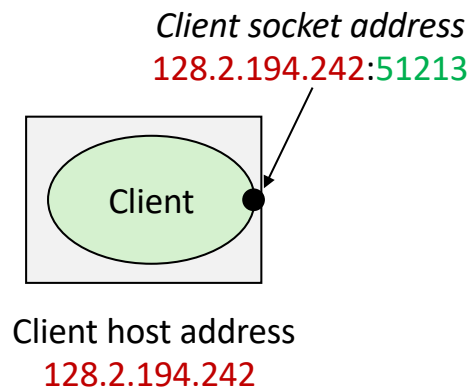
Server host address

208.216.181.15

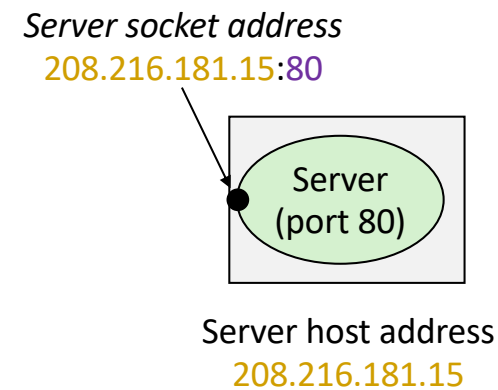
80 is a well-known port  
associated with Web servers

# Anatomy of a Connection

- A connection is uniquely identified by the socket addresses of its endpoints (*socket pair*)
  - `(cliaddr:cliport, servaddr:servport)`



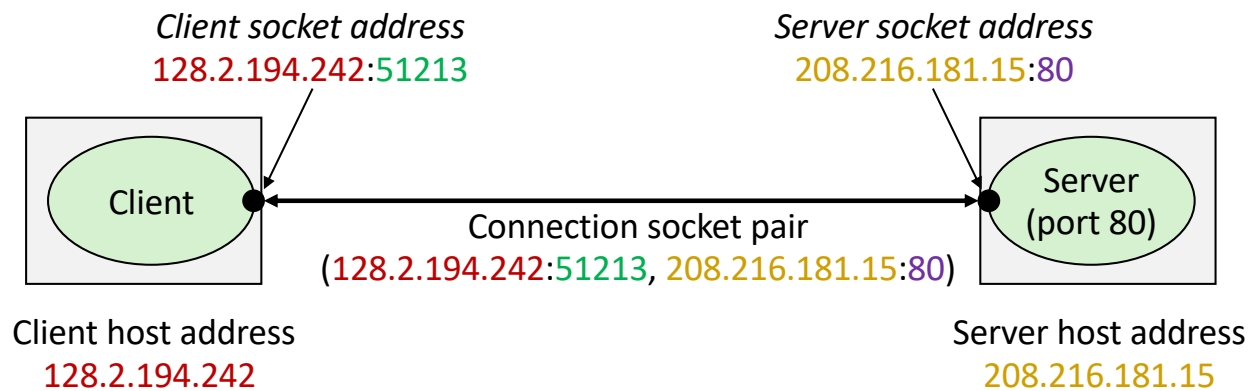
51213 is an ephemeral port  
allocated by the kernel



80 is a well-known port  
associated with Web servers

# Anatomy of a Connection

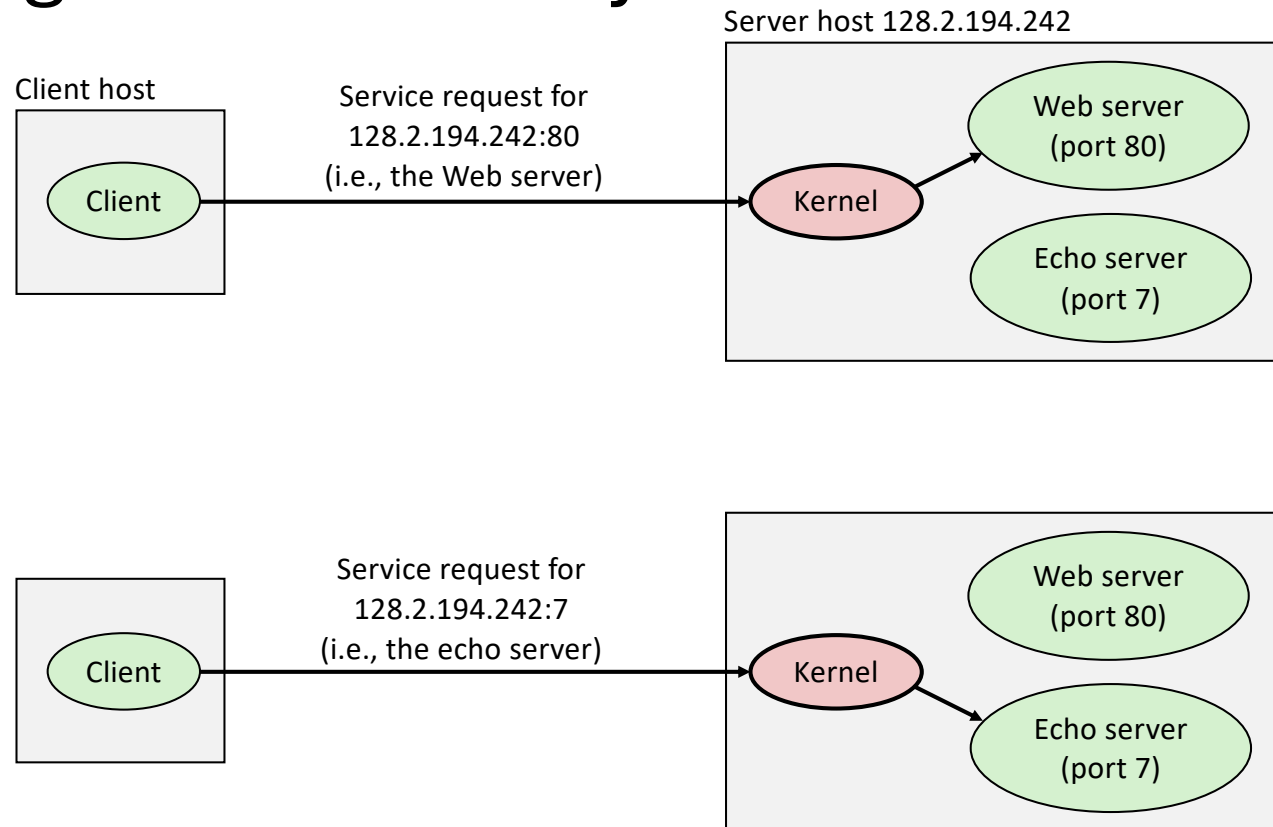
- A connection is uniquely identified by the socket addresses of its endpoints (*socket pair*)
  - `(cliaddr:cliport, servaddr:servport)`



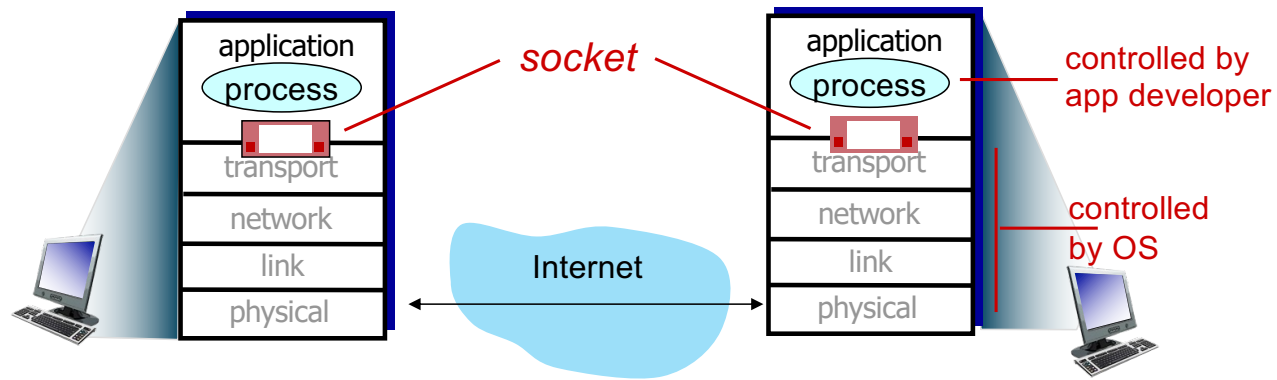
51213 is an ephemeral port  
allocated by the kernel

80 is a well-known port  
associated with Web servers

# Using Ports to Identify Services



# Sockets Interface



# Sockets Interface

- Set of system-level functions used in conjunction with Unix I/O to build network applications.
- Created in the early 80's as part of the original Berkeley distribution of Unix that contained an early version of the Internet protocols.
- Available on all modern systems
  - Unix variants, Windows, OS X, IOS, Android, ARM

# What is a socket?

- To the kernel, a socket is an endpoint of communication
- To an application, a socket is a file descriptor that lets the application read/write from/to the network
  - Using the FD abstraction lets you reuse code & interfaces
- Clients and servers communicate with each other by reading from and writing to socket descriptors



- The main distinction between regular file I/O and socket I/O is how the application “opens” the socket descriptors

# Socket Programming Example

- Echo server and client
- Server
  - Accepts connection request
  - Repeats back lines as they are typed
- Client
  - Requests connection to server
  - Repeatedly:
    - Read line from terminal
    - Send to server
    - Read reply from server
    - Print line to terminal

# Echo Server/Client Session Example

## Server

|   |                                                  |     |
|---|--------------------------------------------------|-----|
| 1 | whaleshark: <i>./echoserveri 6616</i>            |     |
| 2 | Connected to (BAMBOOSHARK.ICS.CS.CMU.EDU, 33707) | (A) |
| 3 | server received 26 bytes                         | (B) |
| 4 | server received 17 bytes                         | (C) |
| 5 | Connected to (BAMBOOSHARK.ICS.CS.CMU.EDU, 33708) | (D) |
| 6 | server received 29 bytes                         | (E) |

**What happens if  
we start client  
before server?**

## Client

|    |                                                                 |     |
|----|-----------------------------------------------------------------|-----|
| 1  | bambooshark: <i>./echoclient whaleshark.ics.cs.cmu.edu 6616</i> | (A) |
| 2  | <i>This line is being echoed</i>                                | (B) |
| 3  | This line is being echoed                                       |     |
| 4  | <i>This one is, too</i>                                         | (C) |
| 5  | This one is, too                                                |     |
| 6  | ^D                                                              |     |
| 7  | bambooshark: <i>./echoclient whaleshark.ics.cs.cmu.edu 6616</i> | (D) |
| 8  | <i>This one is a new connection</i>                             | (E) |
| 9  | This one is a new connection                                    |     |
| 10 | ^D                                                              |     |

# Echo Server/Client Session Example

## Server

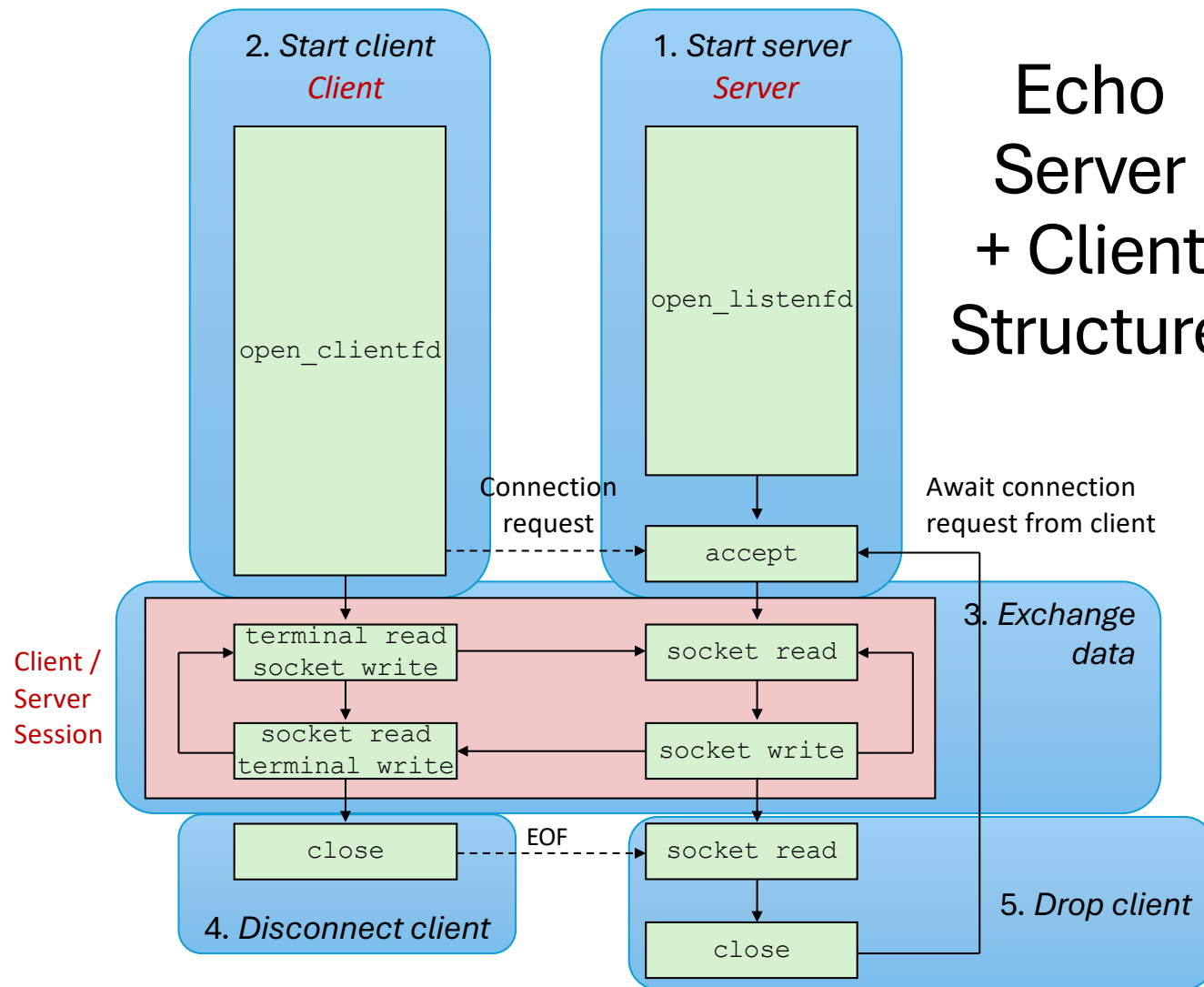
|   |                                                  |     |
|---|--------------------------------------------------|-----|
| 1 | whaleshark: <i>./echoserveri 6616</i>            |     |
| 2 | Connected to (BAMBOOSHARK.ICS.CS.CMU.EDU, 33707) | (A) |
| 3 | server received 26 bytes                         | (B) |
| 4 | server received 17 bytes                         | (C) |
| 5 | Connected to (BAMBOOSHARK.ICS.CS.CMU.EDU, 33708) | (D) |
| 6 | server received 29 bytes                         | (E) |

**Does the server  
stop running  
when we exit  
the client?**

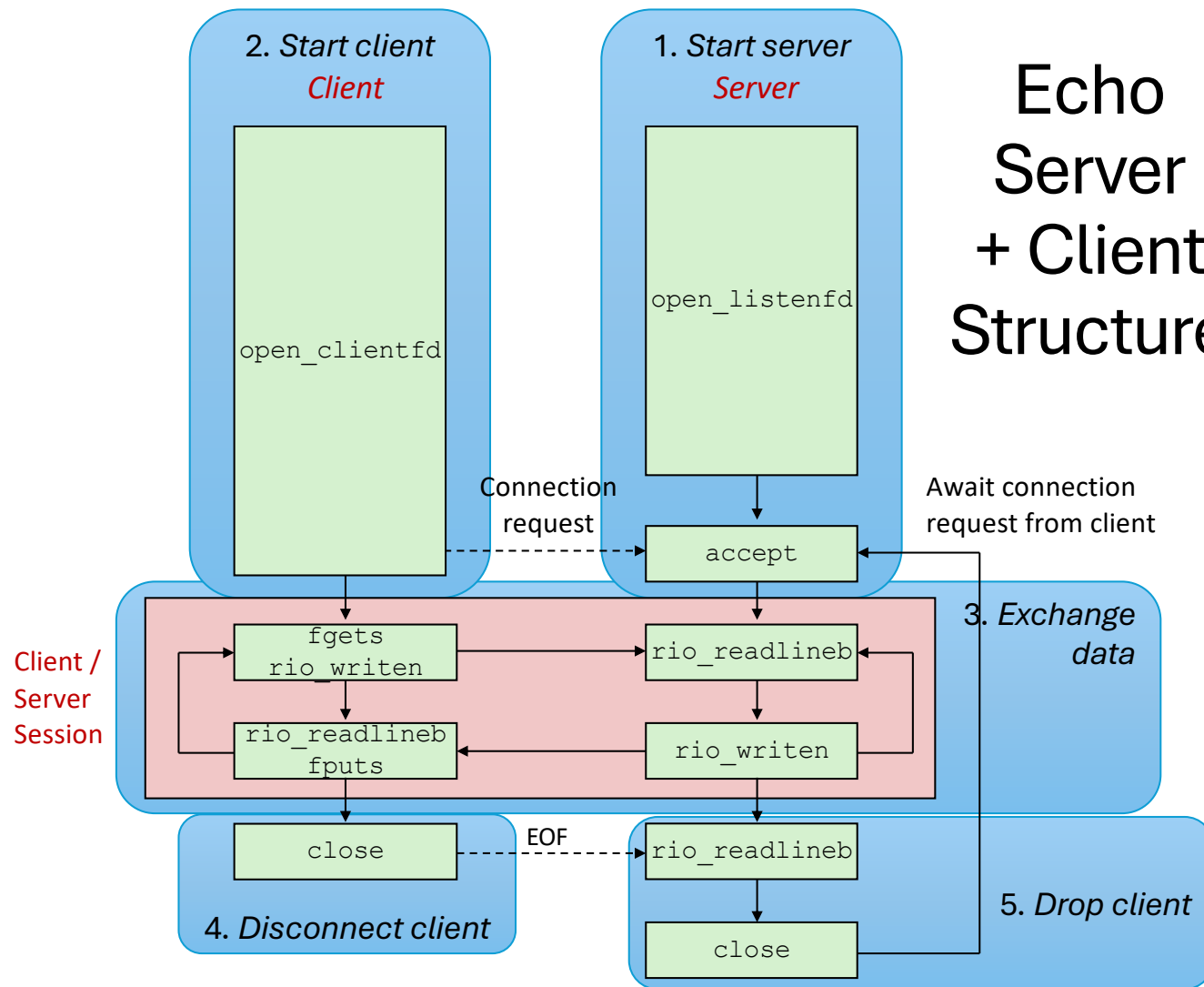
## Client

|    |                                                                 |     |
|----|-----------------------------------------------------------------|-----|
| 1  | bambooshark: <i>./echoclient whaleshark.ics.cs.cmu.edu 6616</i> | (A) |
| 2  | <i>This line is being echoed</i>                                | (B) |
| 3  | This line is being echoed                                       |     |
| 4  | <i>This one is, too</i>                                         | (C) |
| 5  | This one is, too                                                |     |
| 6  | ^D                                                              |     |
| 7  | bambooshark: <i>./echoclient whaleshark.ics.cs.cmu.edu 6616</i> | (D) |
| 8  | <i>This one is a new connection</i>                             | (E) |
| 9  | This one is a new connection                                    |     |
| 10 | ^D                                                              |     |

# Echo Server + Client Structure



# Echo Server + Client Structure



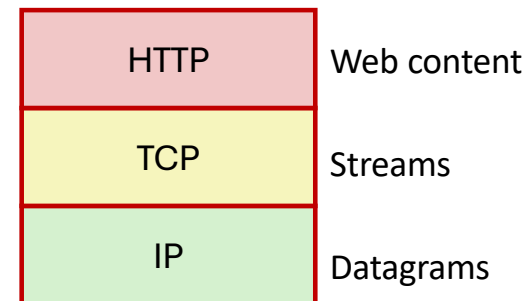
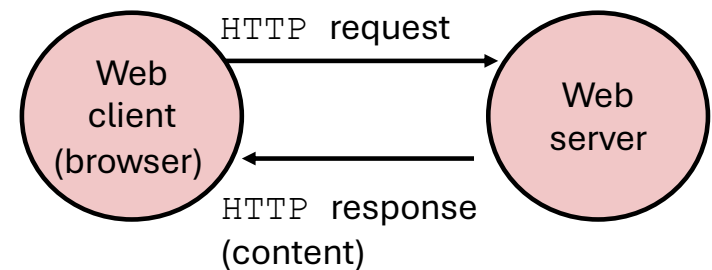
Covering details of echo  
client/server implementation next  
lecture!

# Today: How do computer networks work?

- Network types and structures
- Locating a host
- Setting up a connection
- **HTTP Example**

# Clients and servers communicate using the HyperText Transfer Protocol (HTTP)

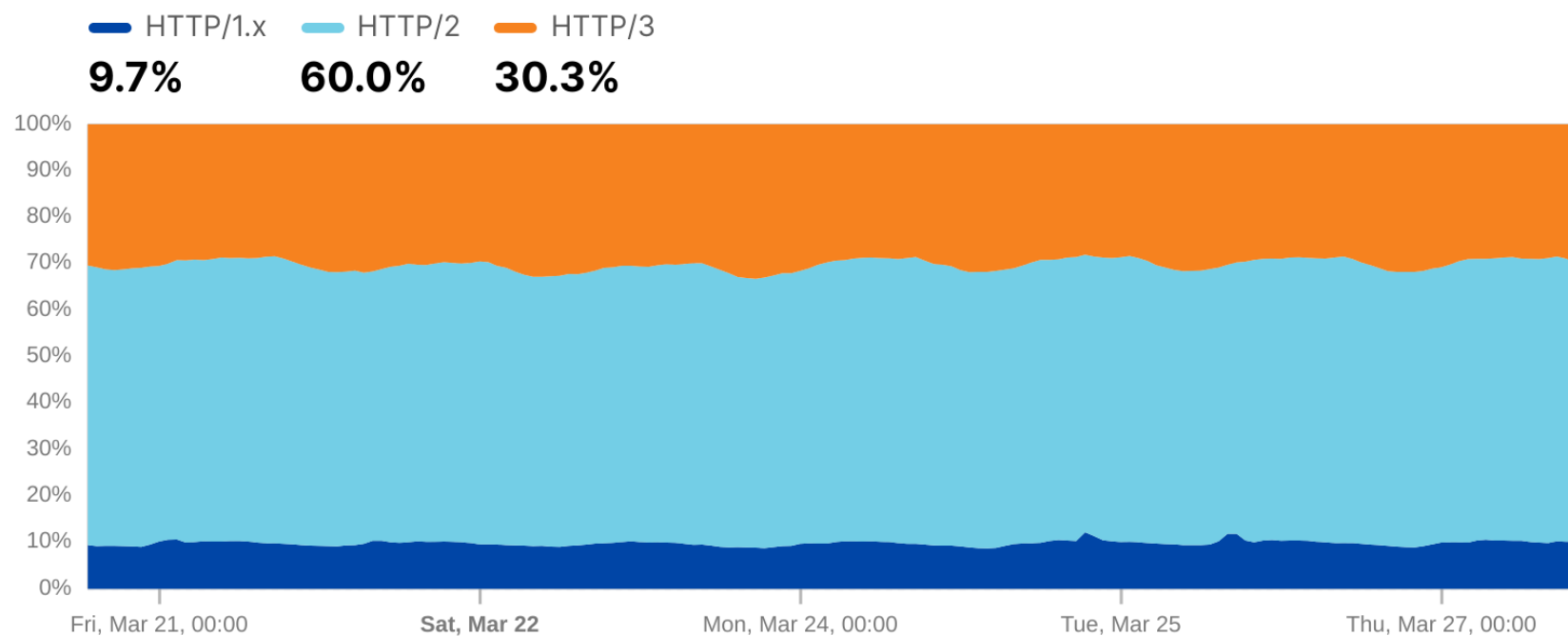
- Client and server establish TCP connection
- Client requests content
- Server responds with requested content
- Client and server close connection (eventually)
- Current version is HTTP/3 but HTTP/1.1 and HTTP/2 widely used still



<https://www.rfc-editor.org/rfc/rfc9110.html>

# HTTP/1.x vs. HTTP/2 vs. HTTP/3 Worldwide

Distribution of human traffic by HTTP version



# Web Content

- Web servers return *content* to clients
  - *content*: a sequence of bytes with an associated MIME (Multipurpose Internet Mail Extensions) type
- Example MIME types
  - `text/html` HTML document
  - `text/plain` Unformatted text
  - `image/gif` Binary image encoded in GIF format
  - `image/png` Binary image encoded in PNG format
  - `image/jpeg` Binary image encoded in JPEG format

You can find the complete list of MIME types at:

<http://www.iana.org/assignments/media-types/media-types.xhtml>

# Static and Dynamic Content

- The content returned in HTTP responses can be either *static* or *dynamic*
  - *Static content*: content stored in files and retrieved in response to an HTTP request
    - Examples: HTML files, images, audio clips, Javascript programs
    - Request identifies which content file
  - *Dynamic content*: content produced on-the-fly in response to an HTTP request
    - Example: content produced by a program executed by the server on behalf of the client
    - Request identifies file containing executable code
- *Web content associated with a file that is managed by the server*

# URLs and how clients and servers use them

- Unique name for a file: URL (Universal Resource Locator)
- Example URL: `http://www.cmu.edu:80/index.html`
- Clients use *prefix* (`http://www.cmu.edu:80`) to infer:
  - What kind (protocol) of server to contact (HTTP)
  - Where the server is (`www.cmu.edu`)
  - What port it is listening on (80)
- Servers use *suffix* (`/index.html`) to:
  - Determine if request is for static or dynamic content.
    - No hard and fast rules for this
    - One convention: executables reside in `cgi-bin` directory
  - Find file on file system
    - Initial “/” in suffix denotes home directory for requested content.
    - Minimal suffix is “/”, which server expands to configured default filename (usually, `index.html`)

# HTTP Request Example

```
GET / HTTP/1.1  
Host: www.cmu.edu
```

```
Client: request line  
Client: required HTTP/1.1 header  
Client: blank line terminates headers
```

- HTTP standard requires that each text line end with `"\r\n"`
- Blank line (`"\r\n"`) terminates request and response headers

# HTTP Requests

- HTTP request is a *request line*, followed by zero or more *request headers*
- Request line: `<method> <uri> <version>`
  - `<method>` is one of **GET**, **POST**, **OPTIONS**, **HEAD**, **PUT**, **DELETE**, or **TRACE**
  - `<uri>` is typically URL for proxies, URL suffix for servers
    - A URL is a type of URI (Uniform Resource Identifier)
    - See <http://www.ietf.org/rfc/rfc2396.txt>
  - `<version>` is HTTP version of request (**HTTP/1.0** or **HTTP/1.1**)
- Request headers: `<header name>: <header data>`
  - Provide additional information to the server

# HTTP Responses

- HTTP response is a *response line* followed by zero or more *response headers*, possibly followed by *content*, with blank line (“\r\n”) separating headers from content.
- Response line:  
`<version> <status code> <status msg>`
  - `<version>` is HTTP version of the response
  - `<status code>` is numeric status
  - `<status msg>` is corresponding English text
    - **200**    **OK**                      Request was handled without error
    - **301**    **Moved**                    Provide alternate URL
    - **404**    **Not found**                Server couldn't find the file
- Response headers: `<header name>: <header data>`
  - Provide additional information about response
  - **Content-Type**: MIME type of content in response body
  - **Content-Length**: Length of content in response body

# Details of client/server implementation

The diagram illustrates the Echo Server + Client Structure, showing the flow of data and control between the client and server components.

**Client Side (Left):**

- 2. Start client Client:** The process begins with `open_clientfd`.
- 3. Exchange data:** Data exchange occurs between the client and server. The client uses `fgets` and `rio_writen` to send data, and `rio_readlineb` and `fputs` to receive data.
- 4. Disconnect client:** The client process ends with `close`.

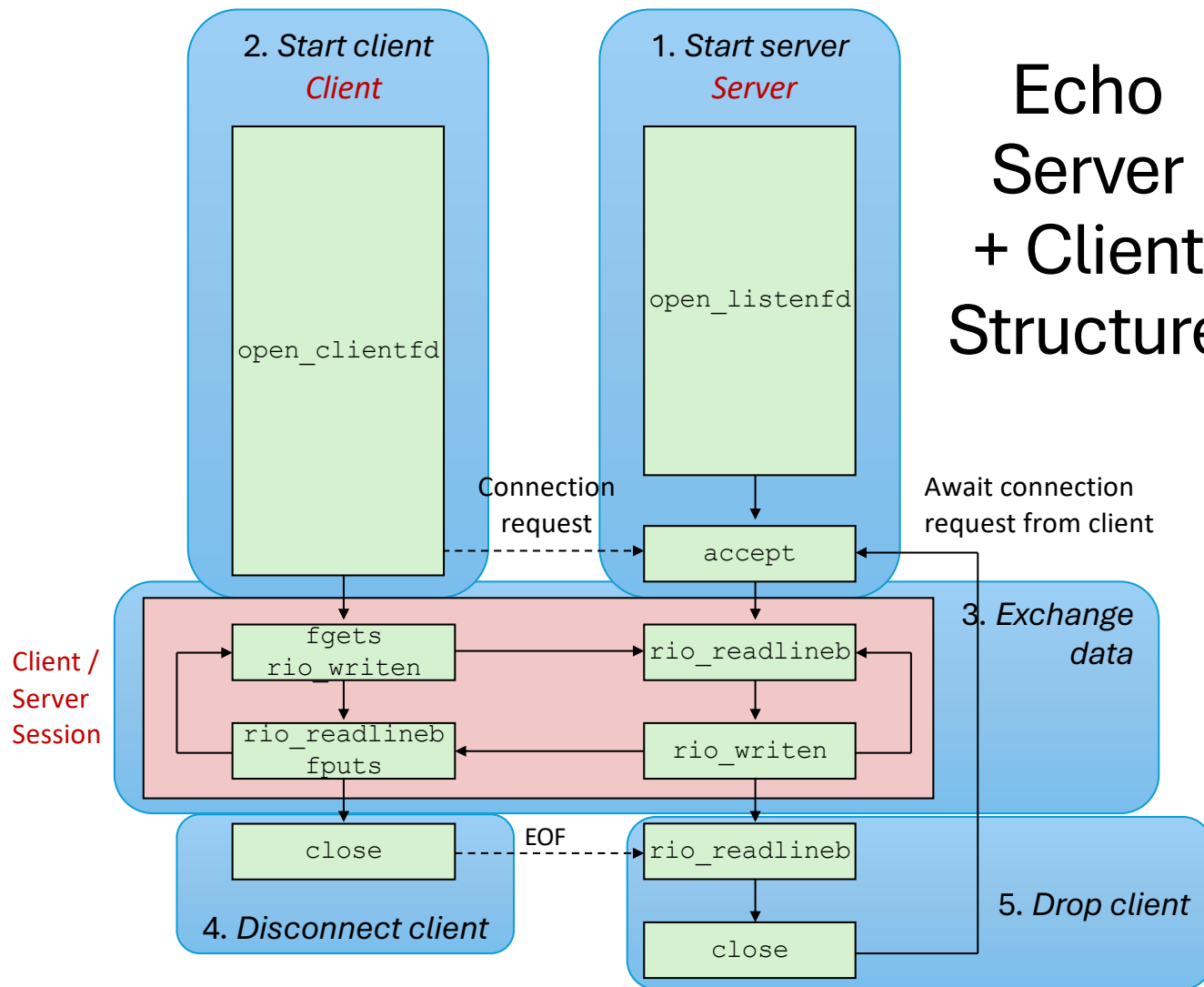
**Server Side (Right):**

- 1. Start server Server:** The process begins with `open_listenfd`.
- 3. Exchange data:** Data exchange occurs between the client and server. The server uses `accept` to receive the connection request, `rio_readlineb` and `rio_writen` to send data, and `rio_readlineb` and `fputs` to receive data.
- 5. Drop client:** The server process ends with `close`.

**Control Flow:**

- A **Connection request** is sent from the client to the server's `accept` function.
- The server **Await connection request from client** and then **Accept connection**.
- EOF** (End of File) is sent from the client's `close` function to the server's `rio_readlineb` function.

**Client / Server Session:** The central area where data exchange and control flow occur, highlighted in a pink box.



# Recall: Unbuffered RIO Input/Output

- Same interface as Unix `read` and `write`
- Especially useful for transferring data on network sockets

```
#include "csapp.h"

ssize_t rio_readn(int fd, void *usrbuf, size_t n);
ssize_t rio_writen(int fd, void *usrbuf, size_t n);
```

Return: num. bytes transferred if OK, 0 on EOF (`rio_readn` only), -1 on error

- **`rio_readn`** returns short count only if it encounters EOF
  - Only use it when you know how many bytes to read
- **`rio_writen`** never returns a short count
- Calls to **`rio_readn`** and **`rio_writen`** can be interleaved arbitrarily on the same descriptor

# Recall: Buffered RIO Input Functions

- Efficiently read text lines and binary data from a file partially cached in an internal memory buffer

```
#include "csapp.h"

void rio_readinitb(rio_t *rp, int fd);

ssize_t rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen);
ssize_t rio_readnb(rio_t *rp, void *usrbuf, size_t n);
```

Return: num. bytes read if OK, 0 on EOF, -1 on error

- **rio\_readlineb** reads a *text line* of up to **maxlen** bytes from file **fd** and stores the line in **usrbuf**
  - Especially useful for reading text lines from network sockets
- Stopping conditions
  - **maxlen** bytes read
  - EOF encountered
  - Newline ('\n') encountered

# Echo Client: Main Routine

```
#include "csapp.h"

int main(int argc, char **argv)
{
    int clientfd;
    char *host, *port, buf[MAXLINE];
    rio_t rio;

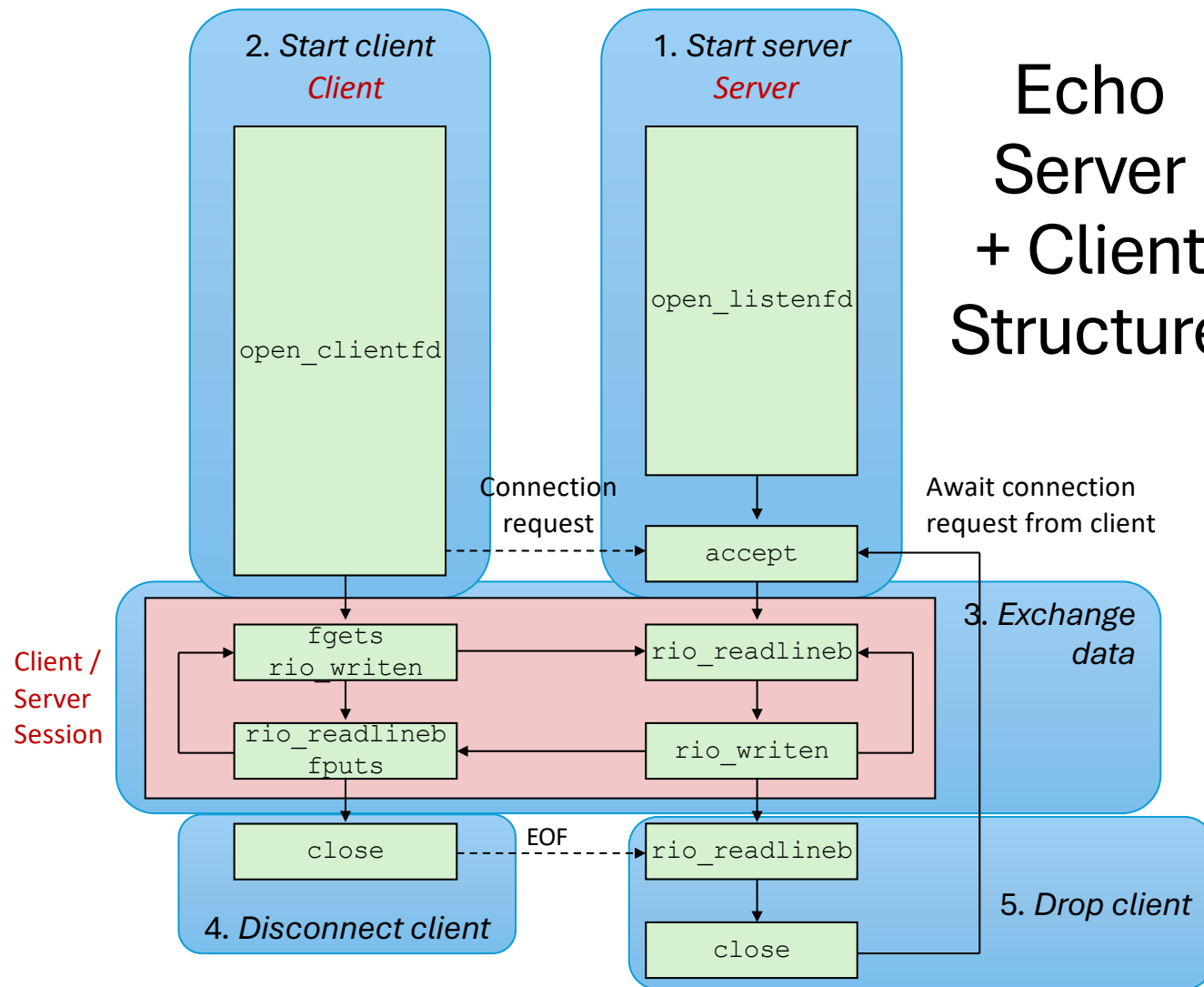
    host = argv[1];
    port = argv[2];

    clientfd = Open_clientfd(host, port);
    Rio_readinitb(&rio, clientfd);

    while (Fgets(buf, MAXLINE, stdin) != NULL) {
        Rio_writen(clientfd, buf, strlen(buf));
        Rio_readlineb(&rio, buf, MAXLINE);
        Fputs(buf, stdout);
    }
    Close(clientfd);
    exit(0);
}
```

echoclient.c

# Echo Server + Client Structure



# Iterative Echo Server: Main Routine

```
#include "csapp.h"
void echo(int connfd);

int main(int argc, char **argv)
{
    int listenfd, connfd;
    socklen_t clientlen;
    struct sockaddr_storage clientaddr; /* Enough room for any addr */
    char client_hostname[MAXLINE], client_port[MAXLINE];

    listenfd = Open_listenfd(argv[1]);
    while (1) {
        clientlen = sizeof(struct sockaddr_storage); /* Important! */
        connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
        Getnameinfo((SA *)&clientaddr, clientlen,
                    client_hostname, MAXLINE, client_port, MAXLINE, 0);
        printf("Connected to (%s, %s)\n", client_hostname, client_port);
        echo(connfd);
        Close(connfd);
    }
    exit(0);
}
```

echoserveri.c

# Echo Server: `echo` function

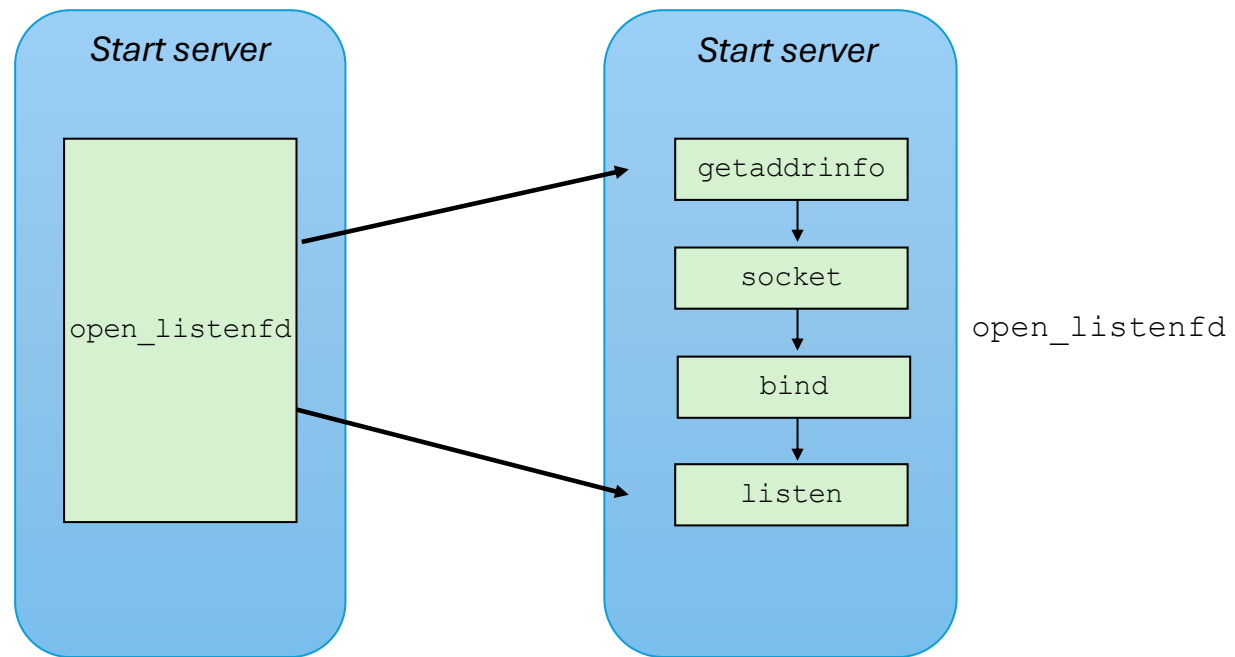
- The server uses RIO to read and echo text lines until EOF (end-of-file) condition is encountered.
  - EOF condition caused by client calling `close(clientfd)`

```
void echo(int connfd)
{
    size_t n;
    char buf[MAXLINE];
    rio_t rio;

    Rio_readinitb(&rio, connfd);
    while((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0) {
        printf("server received %d bytes\n", (int)n);
        Rio_writen(connfd, buf, n);
    }
}
```

echo.c

# Digging Deeper



# Host and Service Conversion:

## `getaddrinfo`

- `getaddrinfo` is the modern way to convert string representations of hostnames, host addresses, ports, and service names to socket address structures.
  - Replaces obsolete `gethostbyname` and `getservbyname` funcs.
- Advantages:
  - Reentrant (can be safely used by threaded programs).
  - Allows us to write portable protocol-independent code
    - Works with both IPv4 and IPv6
- Disadvantages
  - Somewhat complex
  - Fortunately, a small number of usage patterns suffice in most cases.

# Host and Service Conversion: getaddrinfo

```
int getaddrinfo(const char *host,          /* Hostname or address */
                const char *service,      /* Port or service name */
                const struct addrinfo *hints, /* Input parameters */
                struct addrinfo **result); /* Output linked list */

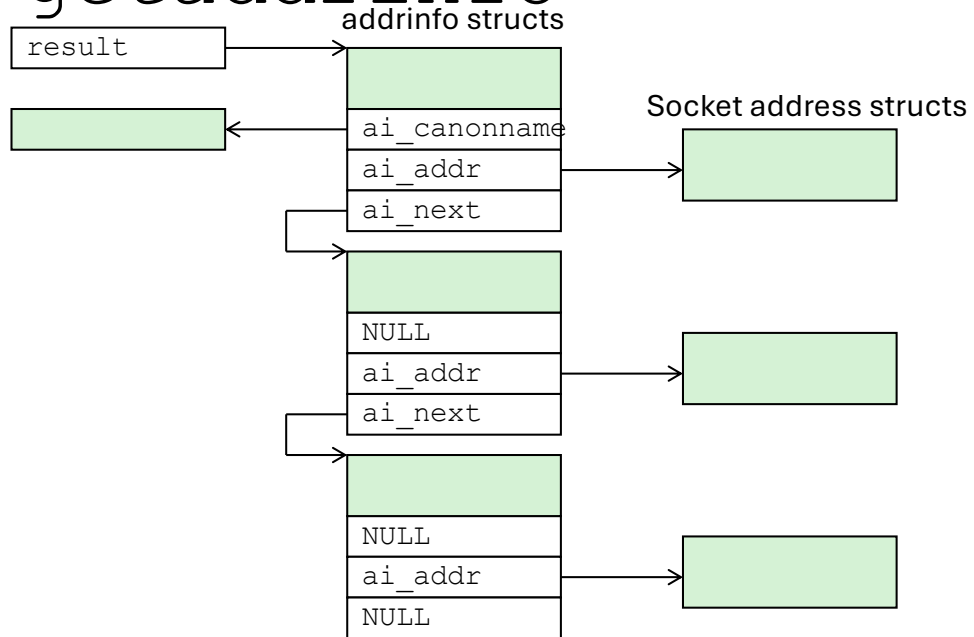
void freeaddrinfo(struct addrinfo *result); /* Free linked list */

const char *gai_strerror(int errcode);     /* Return error msg */
```

Given host and service, getaddrinfo returns result that points to a linked list of **addrinfo** structs, each of which points to a corresponding socket address struct, and which contains arguments for the sockets interface functions.

- Helper functions:
  - freeaddrinfo frees the entire linked list.
  - gai\_strerror converts error code to an error message.

# Linked List Returned by getaddrinfo



# Running hostinfo

```
whaleshark> ./hostinfo localhost  
127.0.0.1
```

```
whaleshark> ./hostinfo whaleshark.ics.cs.cmu.edu  
128.2.210.175
```

```
whaleshark> ./hostinfo twitter.com  
199.16.156.230  
199.16.156.38  
199.16.156.102  
199.16.156.198
```

```
whaleshark> ./hostinfo google.com  
172.217.15.110  
2607:f8b0:4004:802::200e
```

# Sockets Interface: `socket`

- Clients and servers use the `socket` function to create a *socket descriptor*.

- Example:

```
int socket(int domain, int type, int protocol)
```

- Example:

```
int clientfd = socket(AF_INET, SOCK_STREAM, 0);
```

*Protocol specific!*

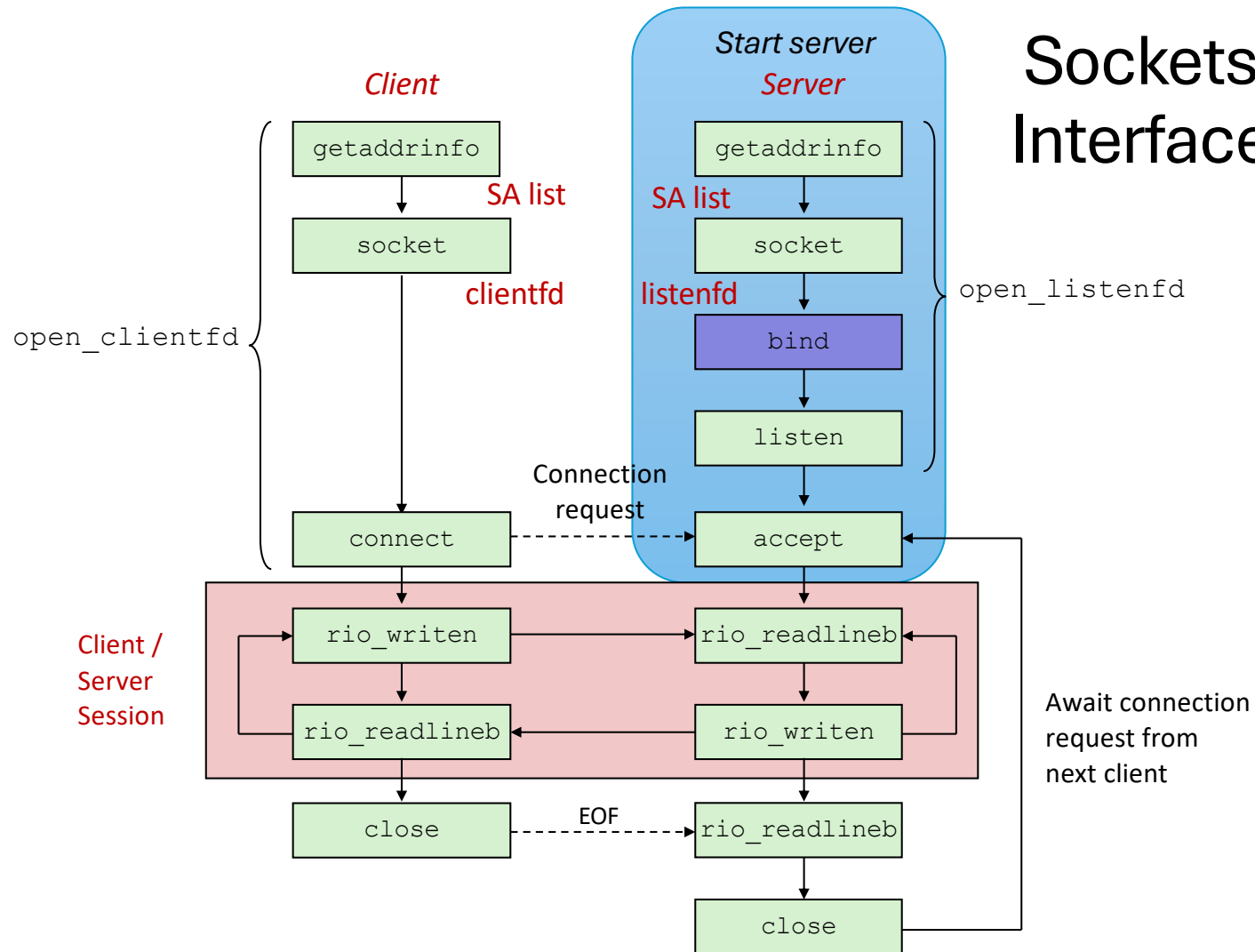
Indicates that we are using  
32-bit IPV4 addresses

Indicates that the socket  
will be the end point of a  
reliable (TCP) connection

```
int clientfd = socket(ai->ai_family, ai->ai_socktype,  
ai->ai_protocol);
```

*Use `getaddrinfo` and you don't have  
to know or care which protocol!*

# Sockets Interface



# Sockets Interface: `bind`

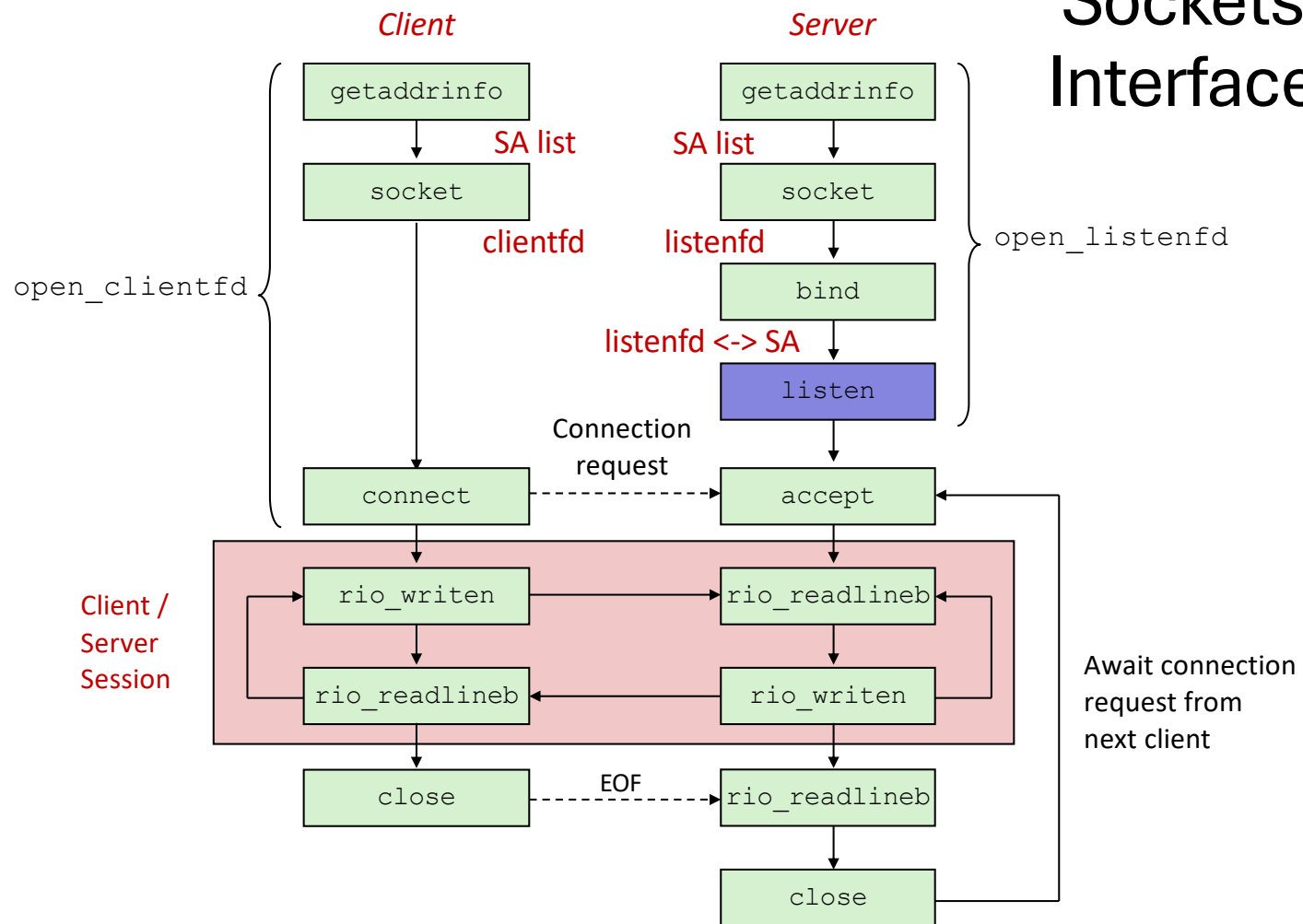
- A server uses `bind` to ask the kernel to associate the server's socket address with a socket descriptor:

Our convention: `typedef struct sockaddr SA;`

- Process can read bytes that arrive on the connection whose endpoint is `addr` by reading from descriptor `sockfd`
- Similarly, writes to `sockfd` are transferred along connection whose endpoint is `addr`

- Best practice: `int bind(int sockfd, SA *addr, socklen_t addrlen);`

# Sockets Interface



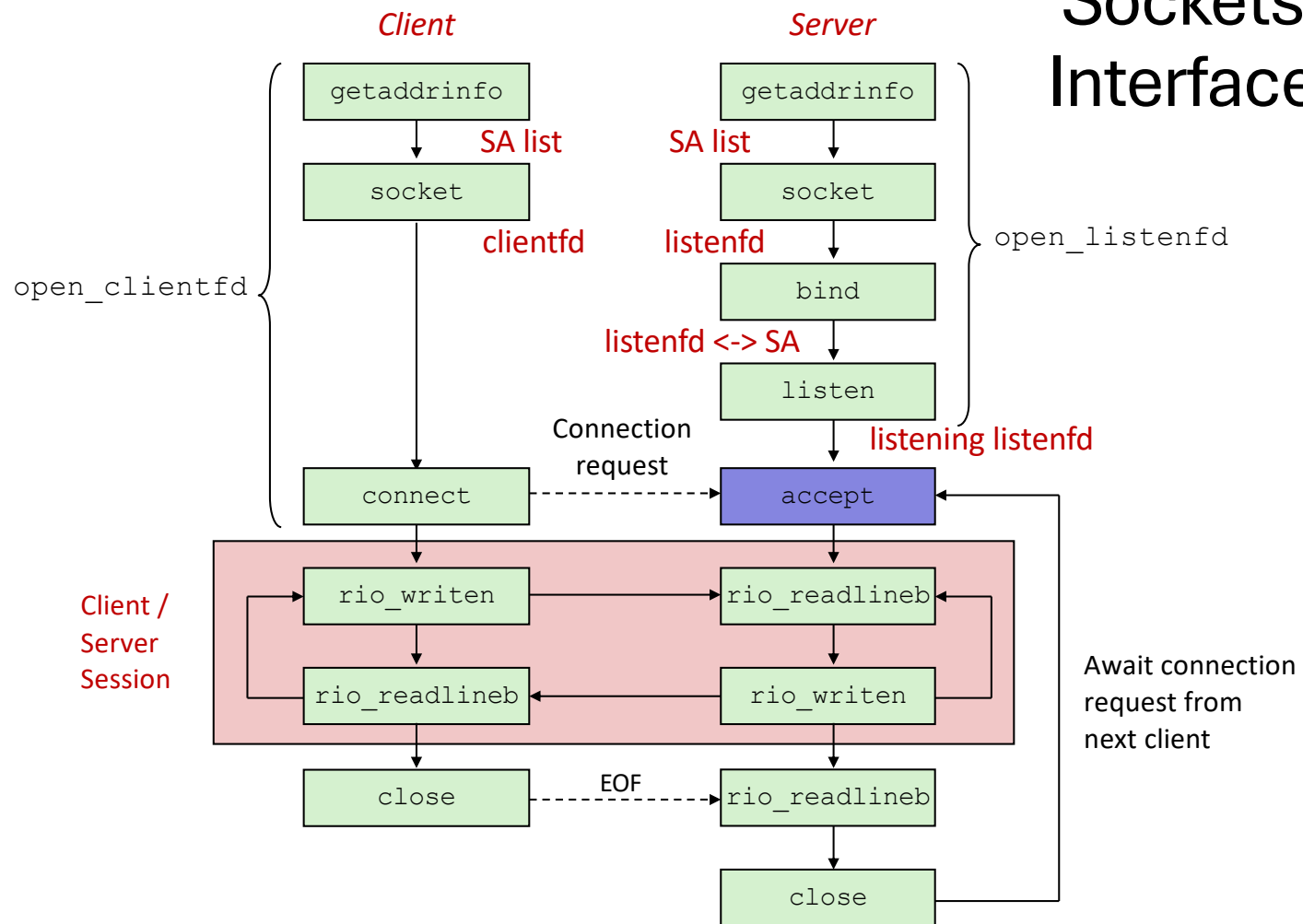
# Sockets Interface: `listen`

- Kernel assumes that descriptor from `socket` function is an *active socket* that will be on the client end
- A server calls the `listen` function to tell the kernel that a descriptor will be used by a server rather than a client:

```
int listen(int sockfd, int backlog);
```

- Converts `sockfd` from an active socket to a *listening socket* that can accept connection requests from clients.
- `backlog` is a hint about the number of outstanding connection requests that the kernel should queue up before starting to refuse requests (128-ish by default)

# Sockets Interface



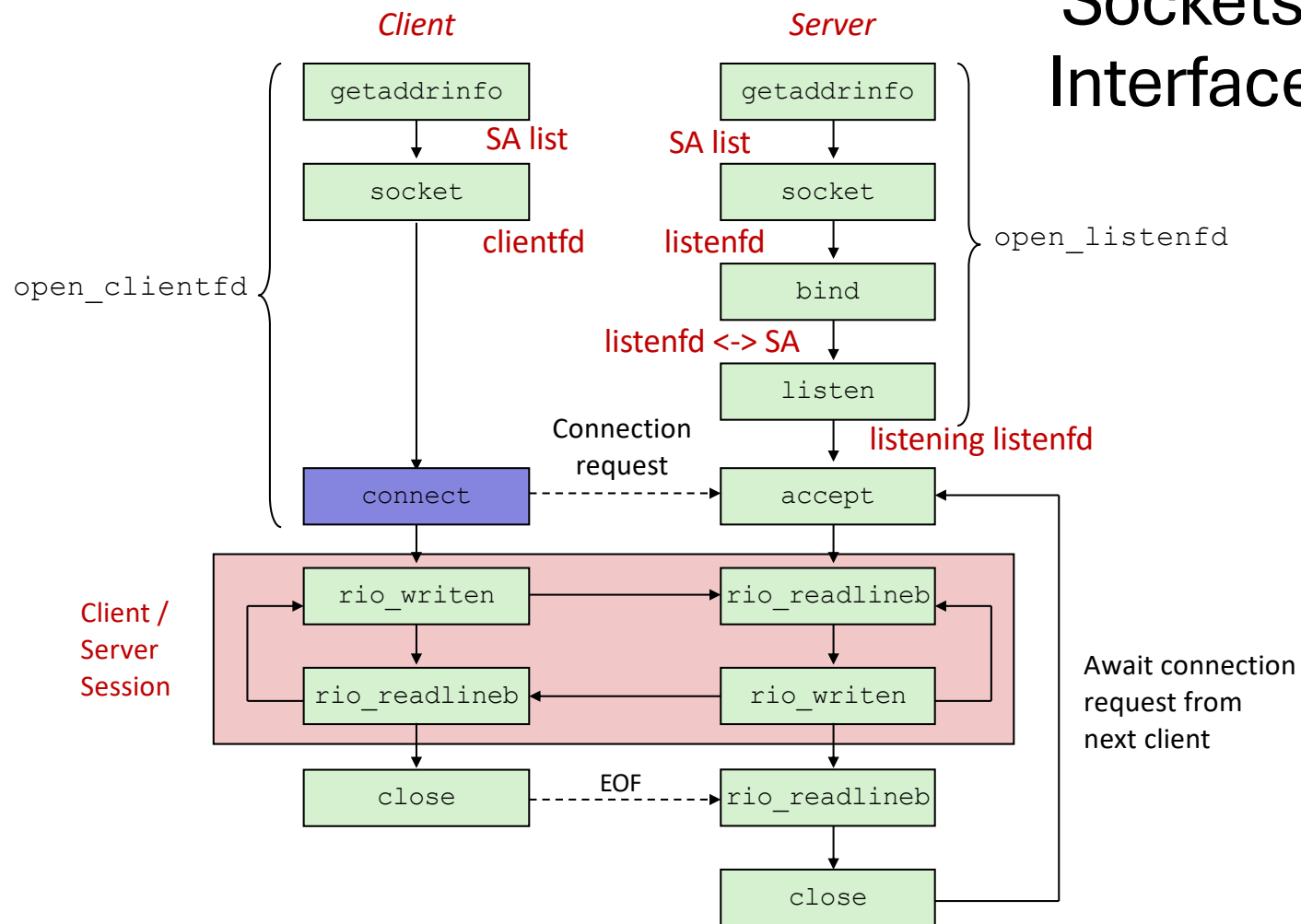
# Sockets Interface: `accept`

- Servers wait for connection requests from clients by calling `accept`:

```
int accept(int listenfd, SA *addr, int *addrlen);
```

- Waits for connection request to arrive on the connection bound to `listenfd`, then fills in client's socket address in `addr` and size of the socket address in `addrlen`.
- Returns a *connected descriptor* `connfd` that can be used to communicate with the client via Unix I/O routines.

# Sockets Interface



# Sockets Interface: connect

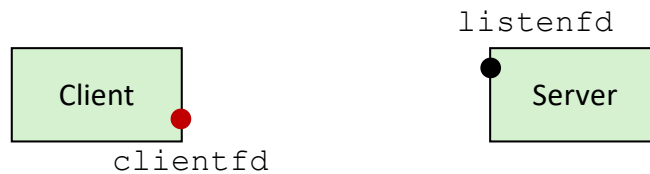
- A client establishes a connection with a server by calling `connect`:

- Attempts to establish a connection with server at socket address `addr`
  - If successful, then `clientfd` is now ready for reading and writing.
  - Resulting connection is characterized by socket pair

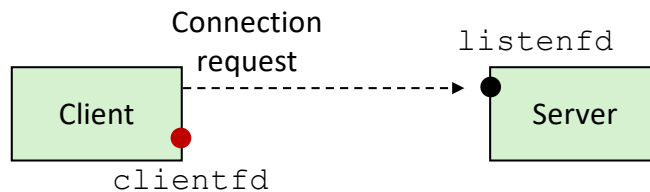
```
int connect(int clientfd, SA *addr, socklen_t addrlen);
```

- Best practice is to use `getaddrinfo` to supply the arguments `addr` and `addrlen`.

# connect/accept Illustrated



1. Server blocks in `accept`, waiting for connection request on listening descriptor `listenfd`



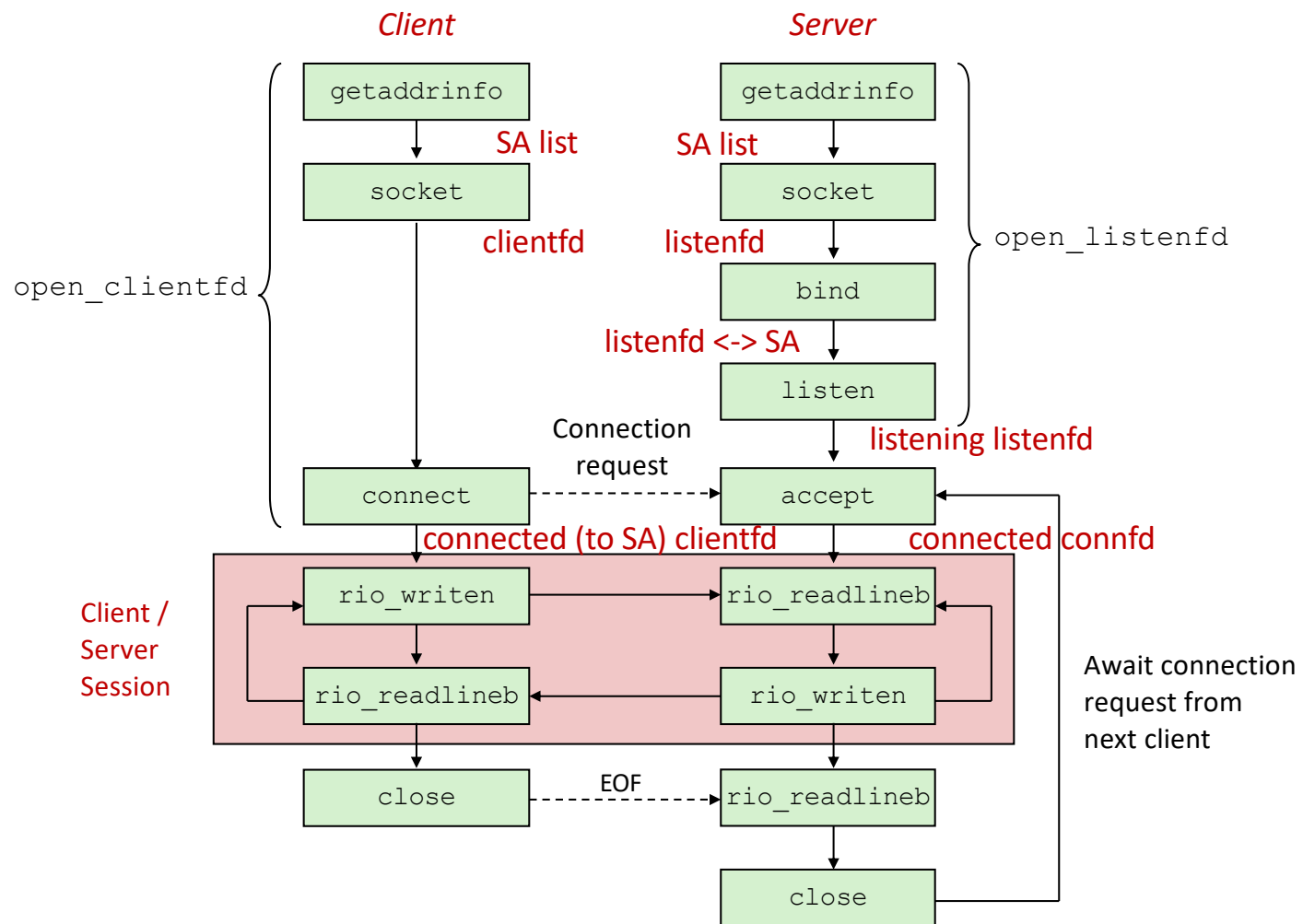
2. Client makes connection request by calling and blocking in `connect`



3. Server returns `connfd` from `accept`. Client returns from `connect`. Connection is now established between `clientfd` and `connfd`

# Connected vs. Listening Descriptors

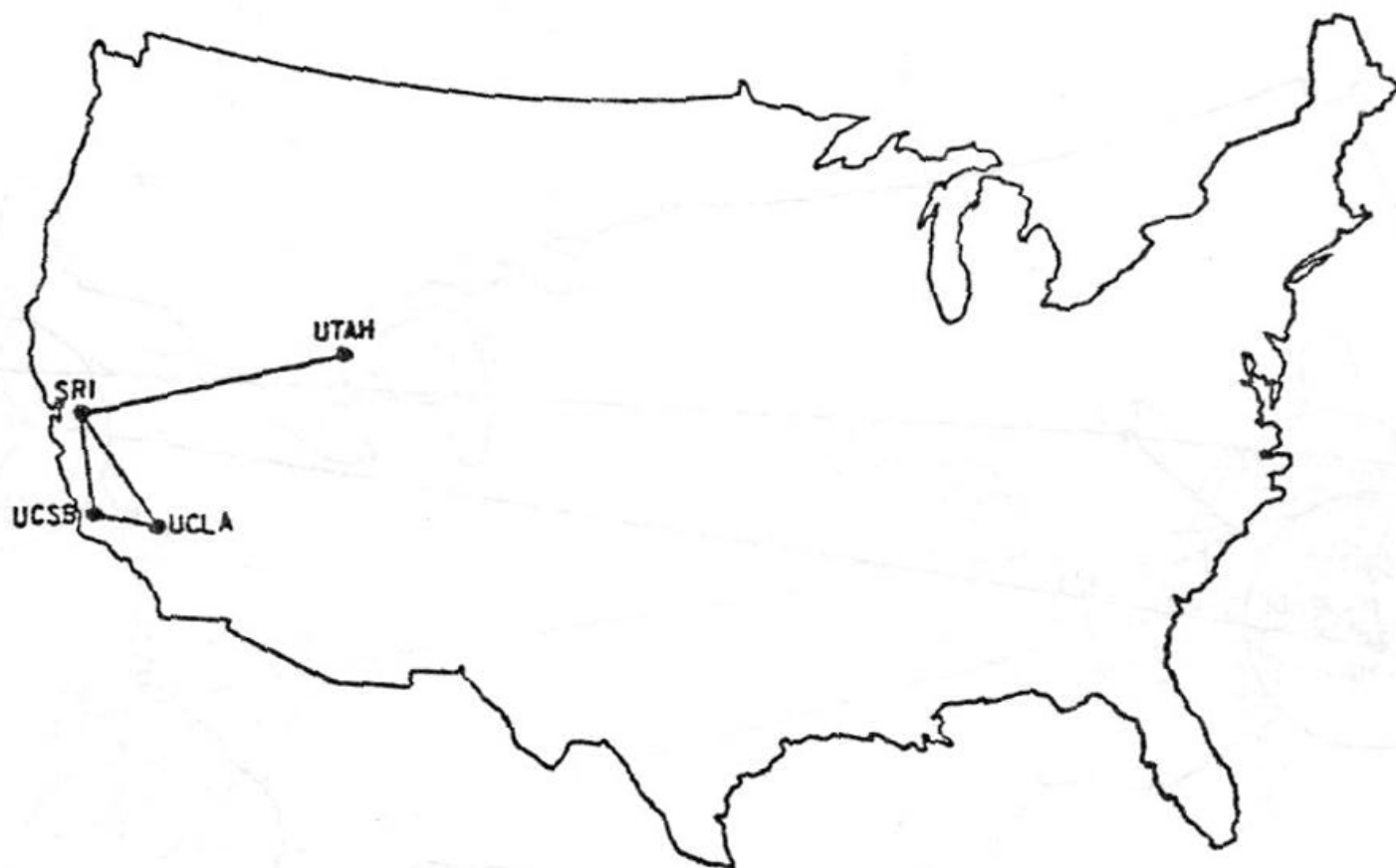
- Listening descriptor
  - End point for client connection requests
  - Created once and exists for lifetime of the server
- Connected descriptor
  - End point of the connection between client and server
  - A new descriptor is created each time the server accepts a connection request from a client
  - Exists only as long as it takes to service client
- Why the distinction?
  - Allows for concurrent servers that can communicate over many client connections simultaneously
    - E.g., Each time we receive a new request, we fork a child to handle the request



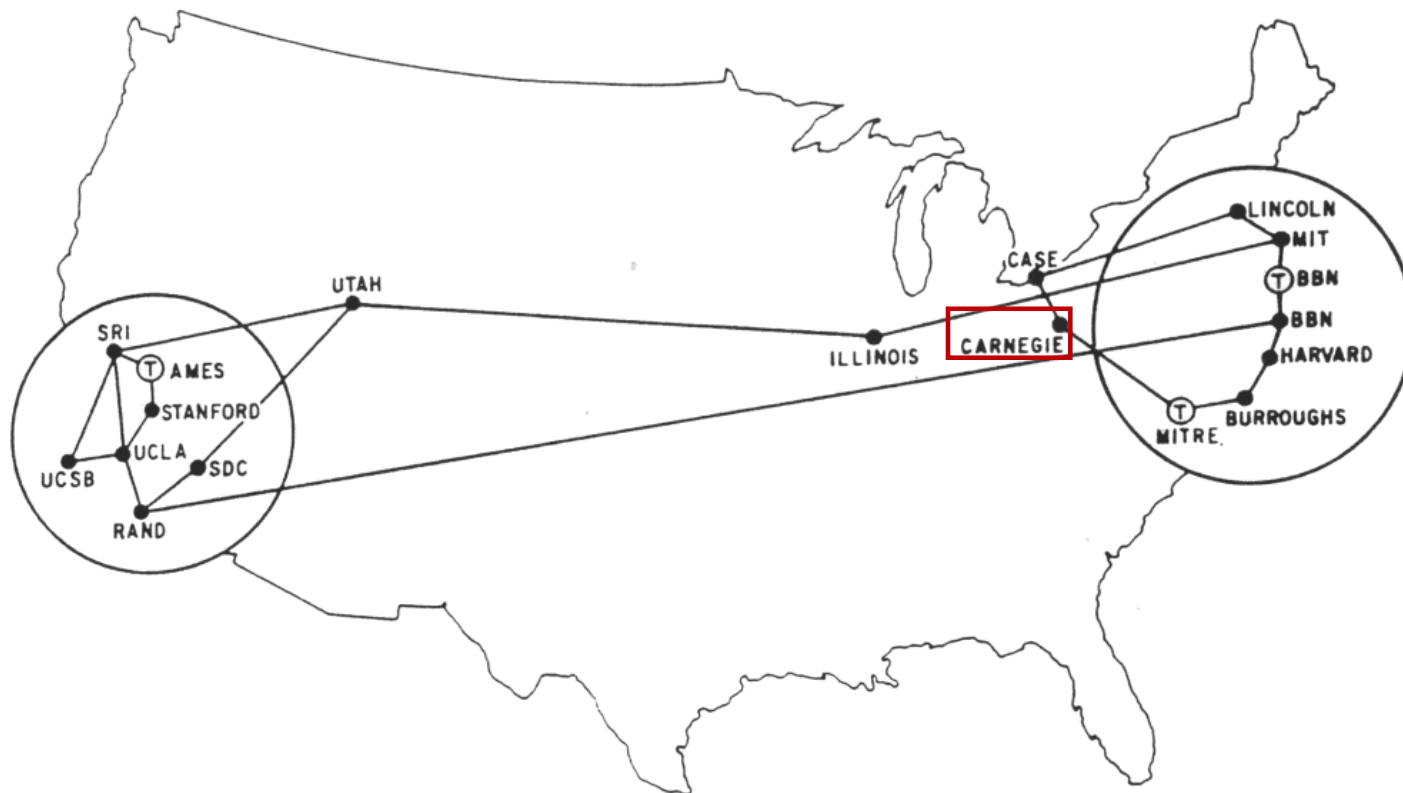
# Outline

- Network types and structures
- Locating a host
- Setting up a connection
- HTTP Example

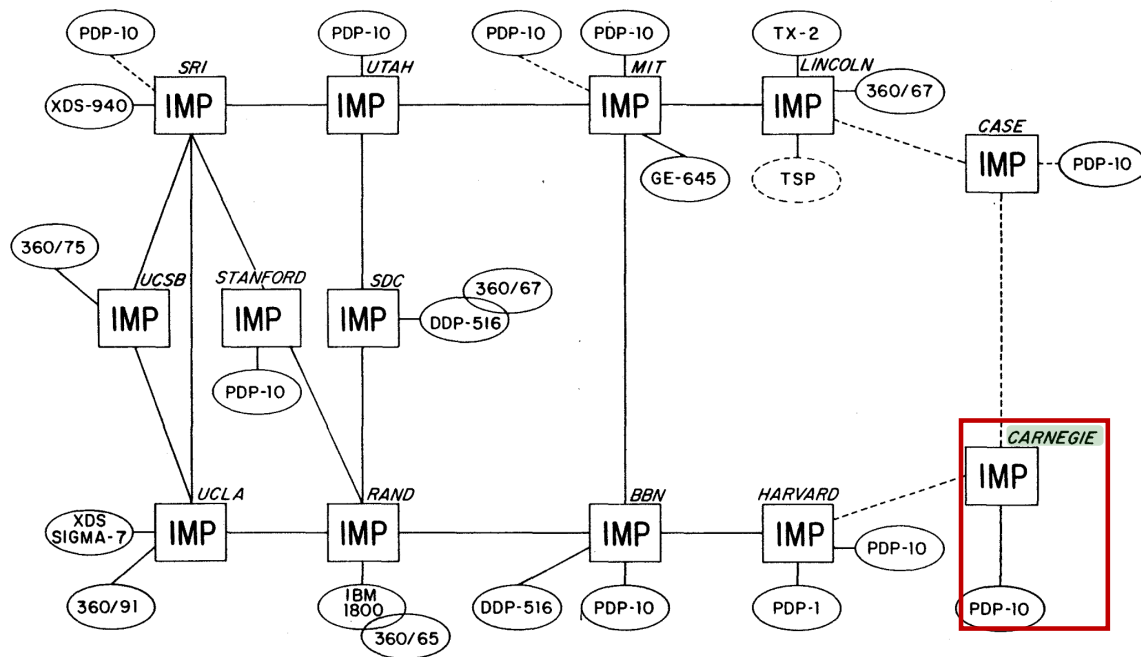
# Appendix



The ARPANET in December 1969

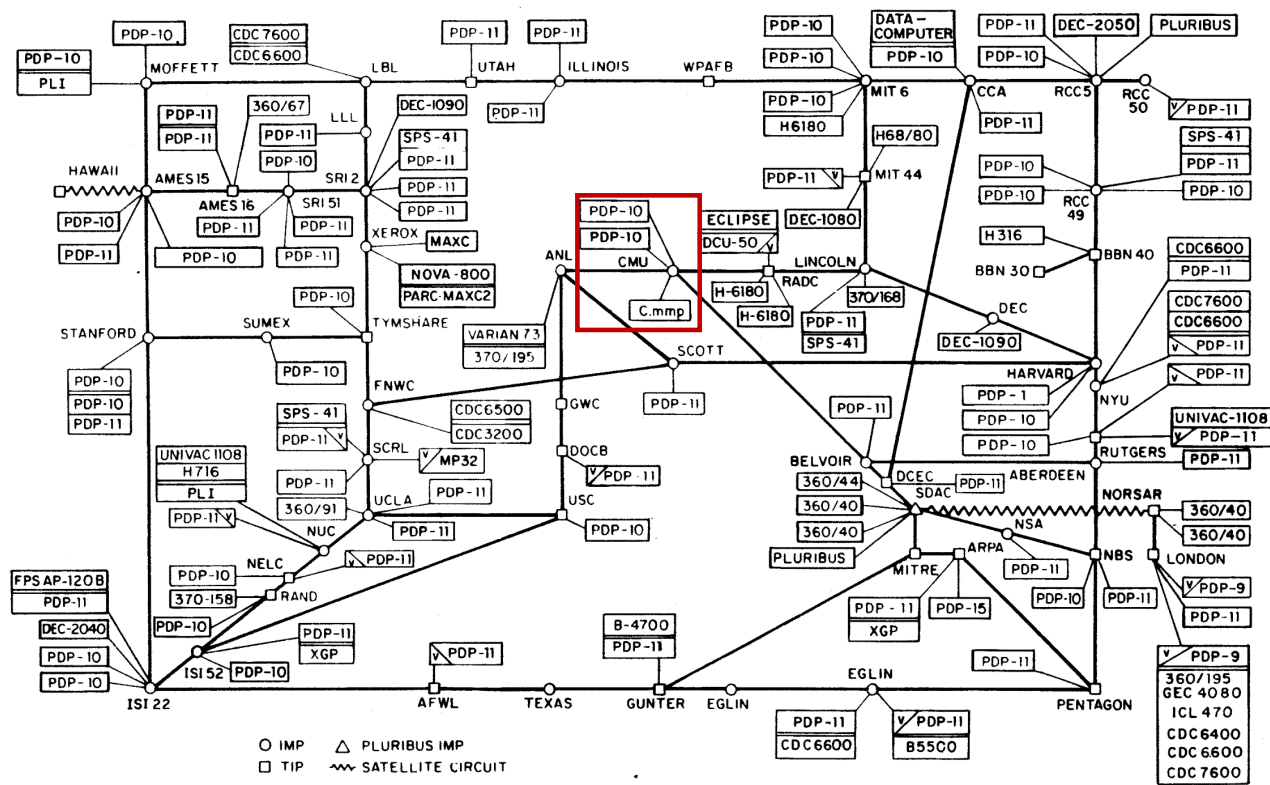


The ARPANET in December 1970



ARPA NET, DECEMBER 1970

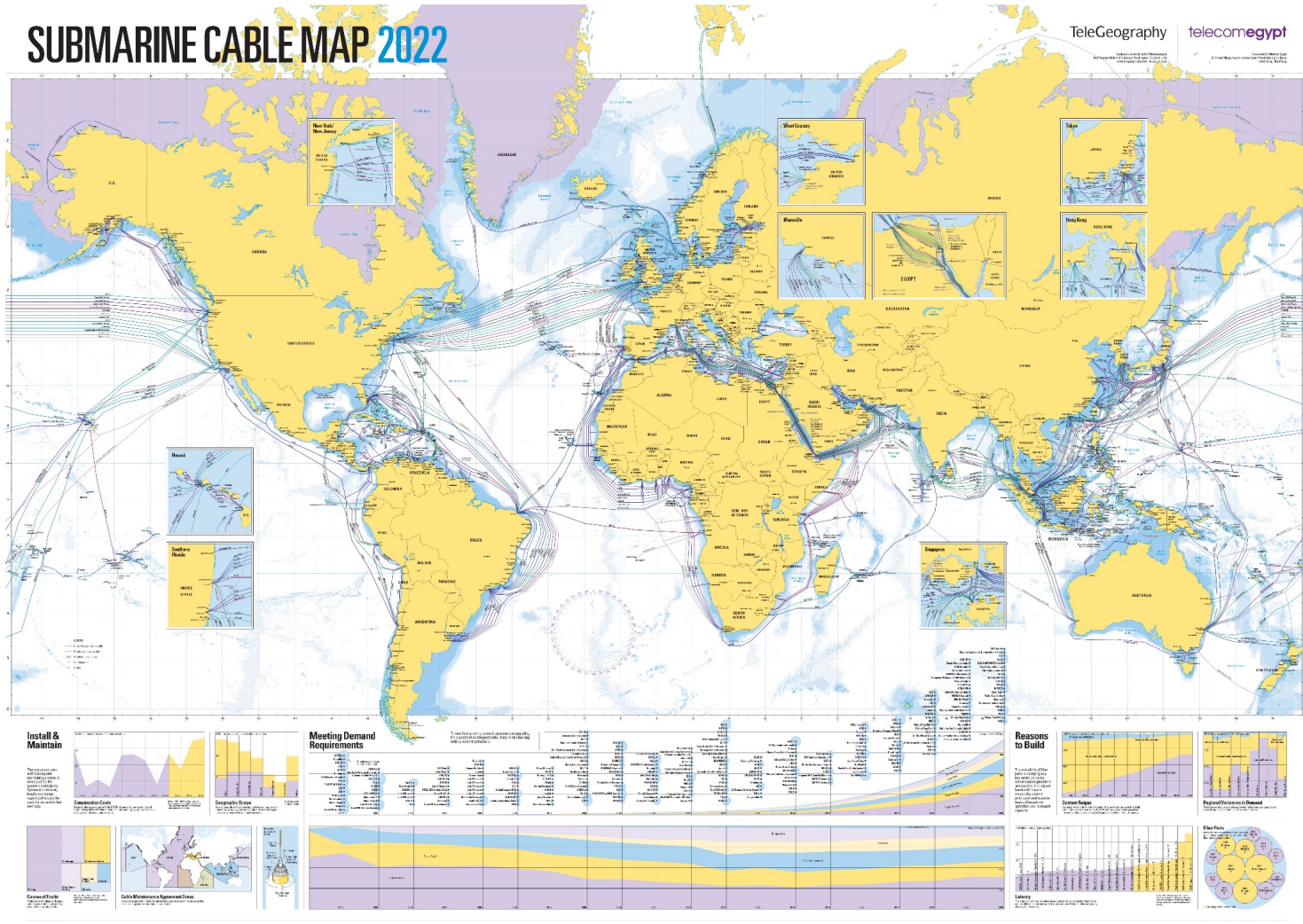
ARPANET LOGICAL MAP, MARCH 1977



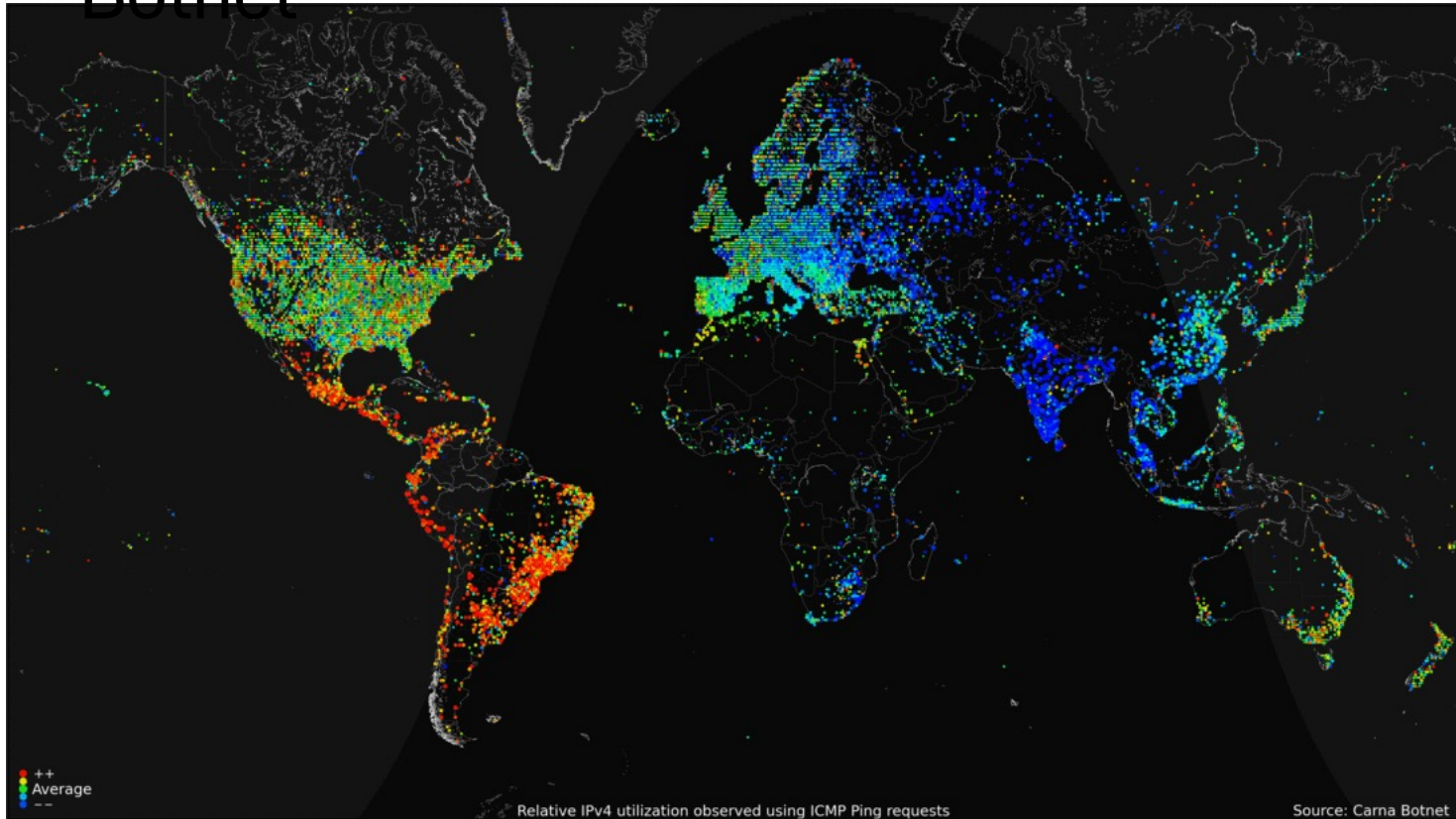
(PLEASE NOTE THAT WHILE THIS MAP SHOWS THE HOST POPULATION OF THE NETWORK ACCORDING TO THE BEST INFORMATION OBTAINABLE, NO CLAIM CAN BE MADE FOR ITS ACCURACY)

# SUBMARINE CABLE MAP 2022

TeleGeography **telecomegypt**



# A Map of 460 Billion Device Connections to the Internet collected by the Carna Botnet



# Basic Internet Components

- Internet backbone:
  - collection of routers (nationwide or worldwide) connected by high-speed point-to-point networks
- Internet Exchange Points (IXP):
  - router that connects multiple backbones (often referred to as peers)
  - Also called Network Access Points (NAP)
- Regional networks:
  - smaller backbones that cover smaller geographical areas (e.g., cities or states)
- Point of presence (POP):
  - machine that is connected to the Internet
- Internet Service Providers (ISPs):
  - provide dial-up or direct access to POPs

# Internet Connection Hierarchy

